

STA891: Prelims Write up

Arijit Dey

Loading necessary libraries:

```
library(tidyverse)
library(purrr)
library(ggplot2)
library(scales)
library(viridis)
library(knitr)
library(rstan)
library(parallel)
library(patchwork)
library(mclust)
library(forcats)
library(scales)
library(rlang)
```

Data Collection steps

We have extracted the data sets from the corresponding websites in the `NHANES.1.R`, `NHEFS.1.R`, `NHEFS.2.R`, `NHEFS.3.R` files that are there in the GitHub repository.

From each of the R file we have one `.Rdata` files containing the raw data files. We load them in the environment as below.

```
# rm(list = ls())

load("NHEFS_1.Rdata")
load("NHEFS_2.Rdata")
load("NHEFS_3.Rdata")
load("NHANES_1.Rdata")
```

```
NHANES.1 <- NHANES.1 |> mutate(wave = "NHANES.1")
NHEFS.1 <- NHEFS.1 |> mutate(wave = "NHEFS.1")
NHEFS.2 <- NHEFS.2 |> mutate(wave = "NHEFS.2")
NHEFS.3 <- NHEFS.3 |> mutate(wave = "NHEFS.3")
```

Data Preparation

To ensure a clean data set, we present the data preparation before delving into any of the any EDA or analysis. This portion of the answer pertains to the question mentioned in 1(d): **Discuss any potential challenges for subsequent analysis based on this exploratory data analysis, including (but not limited to) whether the same type of information is available for each variable at each time point.**

A lot of variables were needed to create, merge, and clean. We painstakingly do them one by one. These would ease up both the EDA and the subsequent analysis.

- For the `vital staus` variable, we could not find it in the NHANES.1 data set. We take everyone to be alive. For the others, we create 2 categories: 1 for alive, 3 for dead. We fill the unknown by NA. We also categorize traced alive but lost prior to interview period, or traced alive but without direct subject contact as alive.

Notably, NHAFES.3 has only alive or dead categories.

```
NHANES.1$vital.st <- 1

NHEFS.1$vital.st <- as.numeric(NHEFS.1$vital.st)
NHEFS.1$vital.st[NHEFS.1$vital.st %in% 4] <- NA
NHEFS.1$vital.st[NHEFS.1$vital.st %in% 5] <- 1

NHEFS.2$vital.st <- as.numeric(NHEFS.2$vital.st)
NHEFS.2$vital.st[NHEFS.2$vital.st %in% 4] <- NA
NHEFS.2$vital.st[NHEFS.2$vital.st %in% c(5,7)] <- 1
```

- For the `sex` variable we just mane it numeric for the four data sets.

```
NHANES.1$sex <- as.numeric(NHANES.1$sex)
NHEFS.1$sex <- as.numeric(NHEFS.1$sex)
NHEFS.2$sex <- as.numeric(NHEFS.2$sex)
NHEFS.3$sex <- as.numeric(NHEFS.3$sex)
```

- For the `race` variable, we have three main categories: White, Black, and Others. NHANES.1 data do so.

For the NHEFS, the try to use finer categories for other, breaking it into Asian, Eskimo or American Indian, and others. We merge them for the NHEFS data set to maintain consistency.

```
NHANES.1$race <- as.numeric(NHANES.1$race)

NHEFS.1$vital.st <- as.numeric(NHEFS.1$vital.st)
NHEFS.1 <- NHEFS.1 |> mutate(race = case_when(race == 4 ~ 1,
                                              race == 3 ~ 2,
                                              race %in% c(1, 2, 5) ~ 3))

NHEFS.2$vital.st <- as.numeric(NHEFS.2$vital.st)
NHEFS.2 <- NHEFS.2 |> mutate(race = case_when(race == 4 ~ 1,
                                              race == 3 ~ 2,
                                              race %in% c(1, 2, 5) ~ 3))

NHEFS.3$vital.st <- as.numeric(NHEFS.3$vital.st)
NHEFS.3 <- NHEFS.3 |> mutate(race = case_when(race == 4 ~ 1,
                                              race == 3 ~ 2,
                                              race %in% c(1, 2, 5) ~ 3))
```

- For the marital status, we create 5 categories: 1 for married, 2 for widowed, 3 for divorced, 4 for separated, and 5 for never married.

Notably, the NHANES cohort contains a bunch of child population, creating a separate category under 17 (marked by 6).

We assign *don't know* or *not ascertained* as NA.

```
NHANES.1$mar.st <- as.numeric(NHANES.1$mar.st)
NHANES.1$mar.st <- NHANES.1$mar.st - 1
NHANES.1$mar.st[NHANES.1$mar.st %in% 0] <- 6
NHANES.1$mar.st[NHANES.1$mar.st %in% 8] <- NA

NHEFS.1$mar.st <- as.numeric(NHEFS.1$mar.st)
NHEFS.1$mar.st[NHEFS.1$mar.st %in% c(8,9)] <- NA

NHEFS.2$mar.st <- as.numeric(NHEFS.2$mar.st)
NHEFS.2$mar.st[NHEFS.2$mar.st %in% c(8)] <- NA
```

```
NHEFS.3$mar.st <- as.numeric(NHEFS.3$mar.st)
NHEFS.3$mar.st[NHEFS.3$mar.st %in% c(-7,-8,-9)] <- NA
```

- We could not find the **General Health** variable in the NHANES.1 data set. We try to fill this column by the entries from another entry: *Have you been bothered by any illness, bodily disorder, pains, or fears about your health during the past month?*, where the entries are 0 for all the time, 1 for most of the time, 2 for a good bit of time, 3 for some of the time, 4 for a little bit of time, and 5 for none of the time.

In the NHEFS data sets the categories are 1 for excellent, 2 for very good, 3 for good, 4 for fair, and 5 for poor.

We map the 6 categories from NHANES to the the NHEFS categories.

```
NHANES.1$gen.hlth <- as.numeric(NHANES.1$gen.hlth)
NHANES.1 <- NHANES.1 |> mutate(gen.hlth =
  case_when(gen.hlth == 5 ~ 1,
            gen.hlth == 4 ~ 2,
            gen.hlth == 3 ~ 3,
            gen.hlth == 2 ~ 4,
            gen.hlth %in% c(1, 0) ~ 5))

NHEFS.1$gen.hlth <- as.numeric(NHEFS.1$gen.hlth)
NHEFS.1$gen.hlth[NHEFS.1$gen.hlth %in% c(1, 7, 8, 9)] <- NA

NHEFS.2$gen.hlth <- as.numeric(NHEFS.2$gen.hlth)
NHEFS.2$gen.hlth[NHEFS.2$gen.hlth %in% c(-1, -7, -8, -9)] <- NA

NHEFS.3$gen.hlth <- as.numeric(NHEFS.3$gen.hlth)
NHEFS.3$gen.hlth[NHEFS.3$gen.hlth %in% c(-1, -7, -8, -9)] <- NA
```

- For the history of heart attack we do cleaning for the unnecessary variables. Notably, for the NHANES data set, we just have one category: 2 for never had condition.

```
NHANES.1$hist.hrt.att <- as.numeric(NHANES.1$hist.hrt.att)

NHEFS.1$hist.hrt.att <- as.numeric(NHEFS.1$hist.hrt.att)
NHEFS.1$hist.hrt.att[NHEFS.1$hist.hrt.att %in% c(8, 9)] <- NA

NHEFS.2$hist.hrt.att <- as.numeric(NHEFS.2$hist.hrt.att)
NHEFS.2$hist.hrt.att[NHEFS.2$hist.hrt.att %in% c(-1, -7, -8, -9)] <- NA
```

```
NHEFS.3$hist.hrt.att <- as.numeric(NHEFS.3$hist.hrt.att)
NHEFS.3$hist.hrt.att[NHEFS.3$hist.hrt.att %in% c(-1, -7, -8, -9)] <- NA
```

- History of TIA couldn't be found for NHANES.1 and NHEFS.3. For the other two data sets, we assign NA for the unnecessary information.

```
NHEFS.1$hist.TIA <- as.numeric(NHEFS.1$hist.TIA)
NHEFS.1$hist.TIA[NHEFS.1$hist.TIA %in% c(8, 9)] <- NA

NHEFS.2$hist.TIA <- as.numeric(NHEFS.2$hist.TIA)
NHEFS.2$hist.TIA[NHEFS.2$hist.TIA %in% c(-1, -8, -9, 96)] <- NA
```

- For the CVA history, we do the same as above. Notably, for the NHANES data set, we just have one category: 2 for never had condition.

```
NHANES.1$hist.CVA <- as.numeric(NHANES.1$hist.CVA)

NHEFS.1$hist.CVA <- as.numeric(NHEFS.1$hist.CVA)
NHEFS.1$hist.CVA[NHEFS.1$hist.CVA %in% c(8, 9)] <- NA

NHEFS.2$hist.CVA <- as.numeric(NHEFS.2$hist.CVA)
NHEFS.2$hist.CVA[NHEFS.2$hist.CVA %in% c(-1, -7, -8, -9)] <- NA

NHEFS.3$hist.CVA <- as.numeric(NHEFS.3$hist.CVA)
NHEFS.3$hist.CVA[NHEFS.3$hist.CVA %in% c(-1, -7, -8, -9)] <- NA
```

- We could not find history of arthritis for the NHEFS.3 data set. For the other NHEFS data sets, we do the similar cleaning.

For the NHANES, there is two extra category: 3 which refers to *had condition, not present now* and 9 which refers to *had condition, do not know if it is present now*. We assign these categories to the category presenting history of arthritis.

```
NHANES.1$hist.arth <- as.numeric(NHANES.1$hist.arth)
NHANES.1$hist.arth[NHANES.1$hist.arth %in% c(3, 9)] <- 1

NHEFS.1$hist.arth <- as.numeric(NHEFS.1$hist.arth)
NHEFS.1$hist.arth[NHEFS.1$hist.arth %in% c(7, 8, 9)] <- NA

NHEFS.2$hist.arth <- as.numeric(NHEFS.2$hist.arth)
NHEFS.2$hist.arth[NHEFS.2$hist.arth %in% c(-1, -7, -8, -9)] <- NA
```

- We could not find history of coronary bypass surgery for the NHANES.1 and NHEFS.1 data set. For the others, we do the similar cleaning.

```
NHEFS.2$hist.cor.byp <- as.numeric(NHEFS.2$hist.cor.byp)
NHEFS.2$hist.cor.byp[NHEFS.2$hist.cor.byp %in% c(-1, -7, -8, -9)] <- NA

NHEFS.3$hist.cor.byp <- as.numeric(NHEFS.3$hist.cor.byp)
NHEFS.3$hist.cor.byp[NHEFS.3$hist.cor.byp %in% c(-1, -7, -8, -9)] <- NA
```

- Unlike NHEFS data sets, NHANES.1 data set had two variables for medication of diabetes, one oral medication and one is insulin. We create a `med.dia` column for NHANES 1 by merging those two columns.

For the other datasets, we clean the unnecessary information.

```
NHANES.1$med.dia.ins <- as.numeric(NHANES.1$med.dia.ins)
NHANES.1$med.dia.oral <- as.numeric(NHANES.1$med.dia.oral)
NHANES.1 <- NHANES.1 |>
  mutate(across(c(med.dia.ins, med.dia.oral), ~ na_if(., 8))) |>
  mutate(med.dia = case_when(
    med.dia.ins == 1 | med.dia.oral == 1 ~ 1,
    med.dia.ins == 2 & med.dia.oral == 2 ~ 2,
    TRUE ~ NA_real_
  )) |>
  select(-c("med.dia.oral", "med.dia.ins"))

NHEFS.1$med.dia <- as.numeric(NHEFS.1$med.dia)
NHEFS.1$med.dia[NHEFS.1$med.dia %in% c(7, 8, 9)] <- NA

NHEFS.2$med.dia <- as.numeric(NHEFS.2$med.dia)
NHEFS.2$med.dia[NHEFS.2$med.dia %in% c(-1, -7, -8, -9, 96)] <- NA

NHEFS.3$med.dia <- as.numeric(NHEFS.3$med.dia)
NHEFS.3$med.dia[NHEFS.3$med.dia %in% c(-1, -7, -8, -9, 96)] <- NA
```

- For the medication regarding high blood pressure, we do the same cleaning stuff for the NHEFS data sets.

For the NHANES, there is an extra category: 3 which refers to *no longer has high blood pressure*. We assign this category to the category presenting not taking medication.

```

NHANES.1$med.hbp <- as.numeric(NHANES.1$med.hbp)
NHANES.1$med.hbp[NHANES.1$med.hbp %in% 3] <- 2

NHEFS.1$med.hbp <- as.numeric(NHEFS.1$med.hbp)
NHEFS.1$med.hbp[NHEFS.1$med.hbp %in% c(7, 8, 9)] <- NA

NHEFS.2$med.hbp <- as.numeric(NHEFS.2$med.hbp)
NHEFS.2$med.hbp[NHEFS.2$med.hbp %in% c(-1, -7, -8, -9, 96)] <- NA

NHEFS.3$med.hbp <- as.numeric(NHEFS.3$med.hbp)
NHEFS.3$med.hbp[NHEFS.3$med.hbp %in% c(-1, -7, -8, -9, 96)] <- NA

```

- We had age and height only for NHANES data set. We can add them Adding age and height on each data sets from the NHEFS data. The **height** variable (in inch) was in the format of 3 digit with the actual weight being XX.X inch. We mutated the column. Further, we roughly add 10, 12, 13 years to the age of a people in NHANES to get the age at three NHEFS data sets, respectively.

```

NHANES.1$age <- as.numeric((NHANES.1$age))
NHANES.1$height <- as.numeric(NHANES.1$height)/10

key_demo <- NHANES.1 |>
  select(ssn, base_age = age, height)

NHEFS.1 <- NHEFS.1 |> left_join(key_demo, by = "ssn")
NHEFS.2 <- NHEFS.2 |> left_join(key_demo, by = "ssn")
NHEFS.3 <- NHEFS.3 |> left_join(key_demo, by = "ssn")

NHEFS.1 <- NHEFS.1 |>
  mutate(age = base_age + 10) |> select(-base_age)

NHEFS.2 <- NHEFS.2 |>
  mutate(age = base_age + 12) |> select(-base_age)

NHEFS.3 <- NHEFS.3 |>
  mutate(age = base_age + 13) |> select(-base_age)

```

- In the NHANES.1, the **weight** variable (in pounds) was in the format of 5 digit with the actual weight being XXX.XX. We mutated the column. For the NHEFS, we had some other entries corresponding to *didn't respond*, *incapable*, etc. We filled them by NA.

```

NHANES.1$weight <- as.numeric(NHANES.1$weight)/100

NHEFS.1$weight <- as.numeric(NHEFS.1$weight)
NHEFS.1$weight[NHEFS.1$weight %in% c(997, 995, 996, 999)] <- NA

NHEFS.2$weight <- as.numeric(NHEFS.2$weight)
NHEFS.2$weight[NHEFS.2$weight %in% c(-1, -7, -8, -9)] <- NA

NHEFS.3$weight <- as.numeric(NHEFS.3$weight)
NHEFS.3$weight[NHEFS.3$weight %in% c(-1, -7, -8, -9)] <- NA

```

- Same goes for the `smoke` variable where there were uninformative entries which we replaced by NA.

```

NHANES.1$smoke <- as.numeric(NHANES.1$smoke)

NHEFS.1$smoke <- as.numeric(NHEFS.1$smoke)
NHEFS.1$smoke[NHEFS.1$smoke %in% c(7, 8, 9)] <- NA

NHEFS.2$smoke <- as.numeric(NHEFS.2$smoke)
NHEFS.2$smoke[NHEFS.2$smoke %in% c(-1, -7, -8, -9)] <- NA

NHEFS.3$smoke <- as.numeric(NHEFS.3$smoke)
NHEFS.3$smoke[NHEFS.3$smoke %in% c(-1, -7, -8, -9)] <- NA

```

- Alcohol history was a bit complicated. First we cleaned the unnecessary information. For the NHANES.1 data the categories are 1 for everyday, 2 for just about everyday, 3 for about 2 or 3 times a week, 4 for about 1-4 times a month, 5 for more than 3 but less than 12 times a year, and 6 for no more than 3 times a year. For the NHEFS.1 data they are 01-31 for number of days in a month, 94 for recently quit drinking, 95 for more than 3 but less than 12 times a year, and 96 for no more than 3 times a year. For NHEFS.2, they are 1-7 for number of days in a week, 94 for more than 3 but less than 12 times a year, and 95 for no more than 3 times a year. For NHEFS.3, they are 1-30 for number of days in a month, 94 for more than 3 but less than 12 times a year, 95 for no more than 3 times a year, and 0 for none. We bring all of them to a common scale with three categories: 1 for frequently, 2 for sometimes, and 3 for never or less than 3 times a year.

```

NHANES.1 <- NHANES.1 |>
  mutate(alcohol = as.numeric(alcohol),
         alcohol = case_when(alcohol == 8 ~ NA_real_,
                             alcohol %in% c(1, 2) ~ 1,
                             alcohol %in% c(3, 4, 5) ~ 2,

```



```

        alcohol == 6 ~ 3,
        TRUE ~ NA_real_))

NHEFS.1 <- NHEFS.1 |>
  mutate(alcohol = as.numeric(alcohol),
         alcohol = case_when(alcohol %in% c(97, 98, 99) ~ NA_real_,
                             alcohol > 10 & alcohol < 32 ~ 1,
                             alcohol < 11 | alcohol == 95 ~ 2,
                             alcohol %in% c(94, 96) ~ 3,
                             TRUE ~ NA_real_))

NHEFS.2 <- NHEFS.2 |>
  mutate(alcohol = as.numeric(alcohol),
         alcohol = case_when(alcohol %in% c(97, 98, 99) ~ NA_real_,
                             alcohol > 2 & alcohol < 8 ~ 1,
                             alcohol < 3 | alcohol == 94 ~ 2,
                             alcohol == 95 ~ 3,
                             TRUE ~ NA_real_))

NHEFS.3 <- NHEFS.3 |>
  mutate(alcohol = as.numeric(alcohol),
         alcohol = case_when(alcohol %in% c(97, 98, 99) ~ NA_real_,
                             alcohol %in% c(0, 95) ~ 3,
                             alcohol > 10 & alcohol < 31 ~ 1,
                             alcohol < 11 | alcohol == 94 ~ 2,
                             TRUE ~ NA_real_))

```

Description of the data

Based on the data preparation above, we describe the variables with the corresponding categories as below.

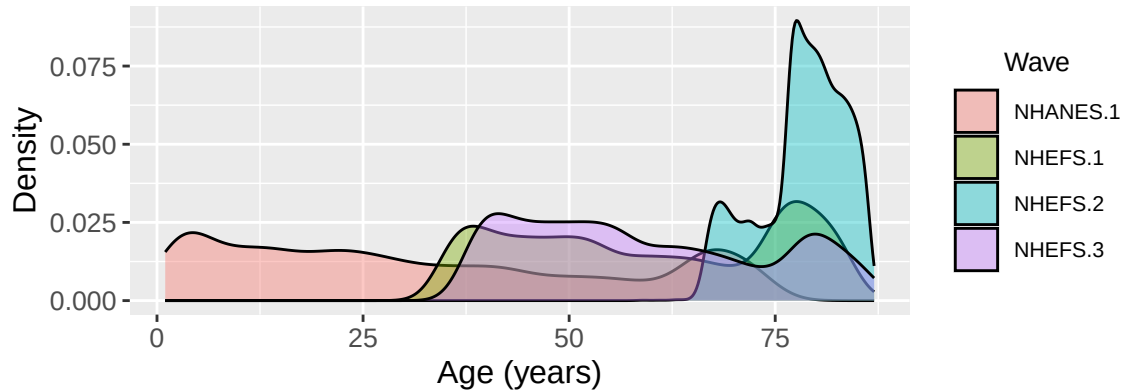
Table 1: Variables and their descriptions

Variable	Description
ssn	Participant identifier
age	Age (years) at the given wave
sex	Sex (1: male, 2: female)
race	Race (1: white, 2: black, 3: Others)
mar.st	Marital status (1: married, 2: widowed, 3: divorced, 4: separated, 5: never married, and 6: Under 17)
vital.st	Vital status at follow-up (1: alive, 3: dead)
gen.hlth	Self-reported general health status (1: excellent, 2: very good, 3: good, 4: fair, and 5: poor)
height	Baseline height (inches)
weight	Weight at given wave (pounds)
smoke	Current smoking status (1: Yes, 2: No)
alcohol	Alcohol use status (1: Frequently, 2: Sometimes, 3: Almost Never)
hist.arth	History of arthritis (1: Yes, 2: No)
hist.hrt.att	History of heart attack (1: Yes, 2: No)
hist.TIA	History of transient ischemic attack (TIA) (1: Yes, 2: No)
hist.CVA	History of cerebrovascular accident (stroke) (1: Yes, 2: No)
hist.cor.byp	History of coronary bypass surgery (1: Yes, 2: No)
med.hbp	Takes medication for high blood pressure (1: Yes, 2: No)
med.dia	Takes medication for diabetes (1: Yes, 2: No)

EDA

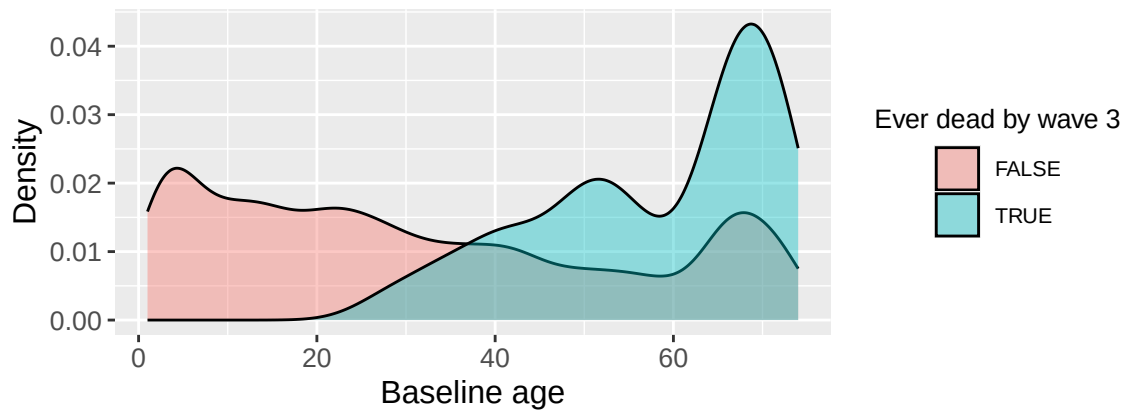
We provide a handful of plots exploratory data analysis. Almost in all the plots we present instances of each wave for easy comparison.

1. Age distribution by wave



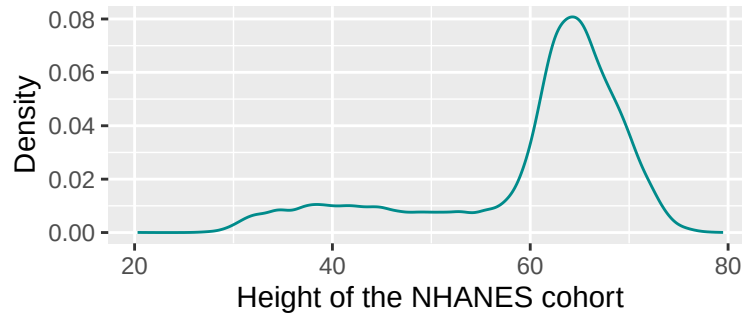
The age distributions shift markedly across waves: NHANES.1 includes many younger participants, while NHEFS.1–3 are increasingly dominated by older adults, with NHEFS.3 concentrated in the 60–80 range. This reflects natural aging and selective loss to follow-up or death, so any longitudinal comparisons or clustering across waves must account for this strong change in age structure and potential survivorship bias.

2. Baseline age distribution of people who were eventually dead



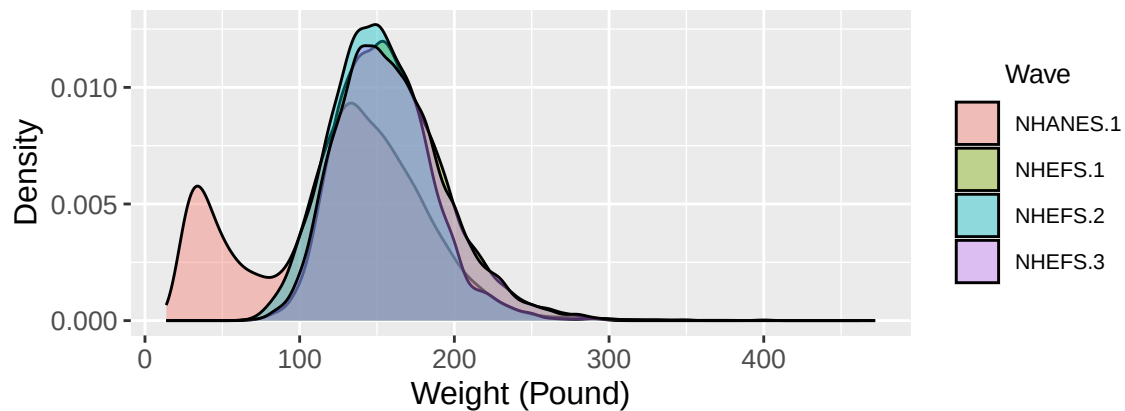
The baseline age distributions differ sharply by eventual vital status: those who died by wave 3 are overwhelmingly older at baseline, with density concentrated around the 60–70 age range, whereas survivors are much younger on average and have substantial mass in childhood and early adulthood. This highlights strong age-related mortality and underscores the importance of adjusting for baseline age when interpreting longitudinal and clustering results.

3. Distribution of height of the participants



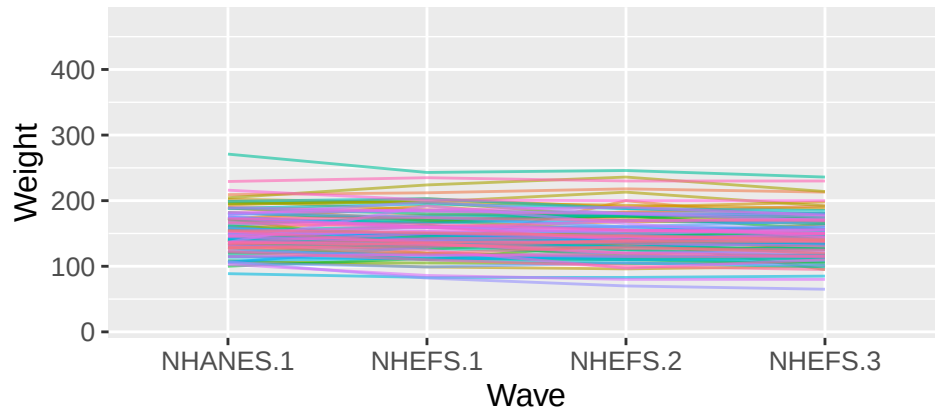
The above shows the kernel density of height for the NHANES cohort. We see that there are a lot of participants with shorter height indicating the prevalence of children. The height distributions for all survey waves are practically identical, so their density curves overlap and appear as a single curve in the plot.

4. Weight distribution by wave



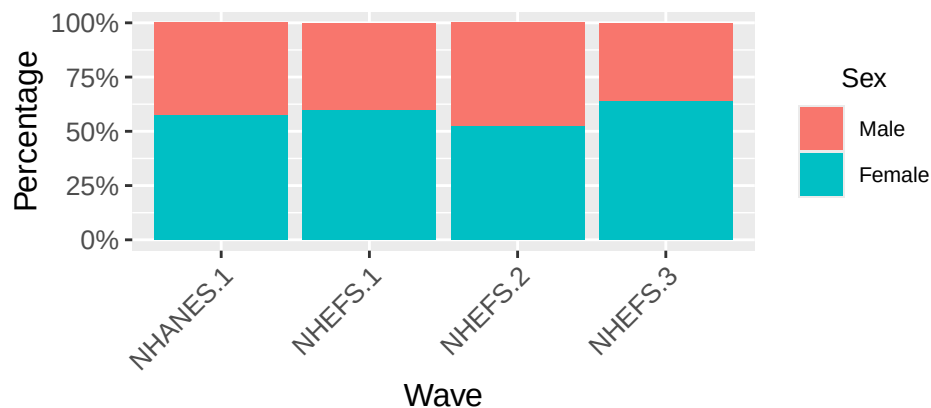
The weight distributions show that NHANES.1 contains a lighter subgroup (children/adolescents) that disappears in the follow-up waves, which are concentrated in the adult range around 130–200 pounds. Across NHEFS.1–3 the main peak remains similar, suggesting only modest shifts in average weight over time relative to the strong compositional change from including vs. excluding younger participants.

5. Weight trajectories for a random sample of 100 people



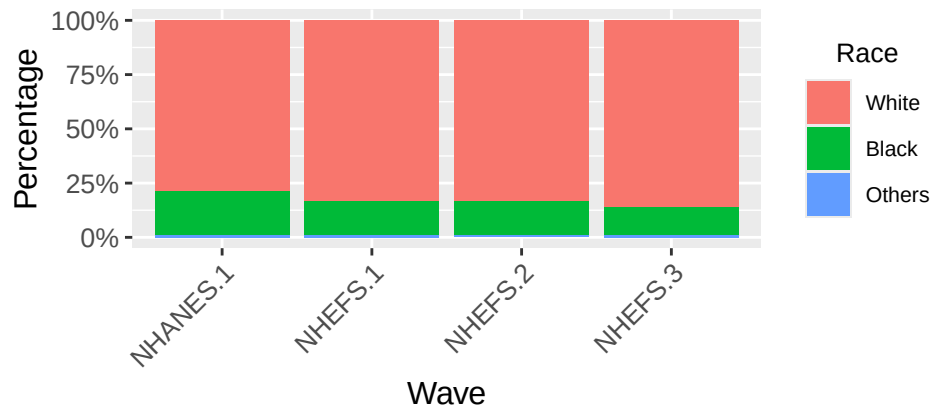
For this random sample of 100 participants, individual weight trajectories appear largely stable across waves: most lines fluctuate only modestly around their baseline values, with no systematic upward or downward trend. A few individuals show larger gains or losses, but there is no evidence of widespread drastic weight change in the cohort over time.

6. Sex-ratio for the four waves



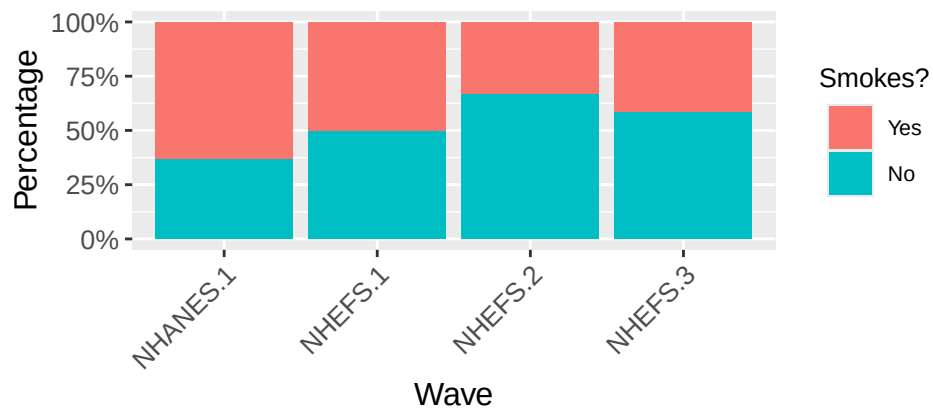
The sex distribution is fairly stable across waves, with men and women represented in roughly similar proportions at each time point. There is a slight shift toward a higher proportion of women in NHEFS.3, suggesting that men may be somewhat more likely to die or be lost to follow-up, but overall the cohort remains broadly balanced by sex.

7. Race categories for the four waves



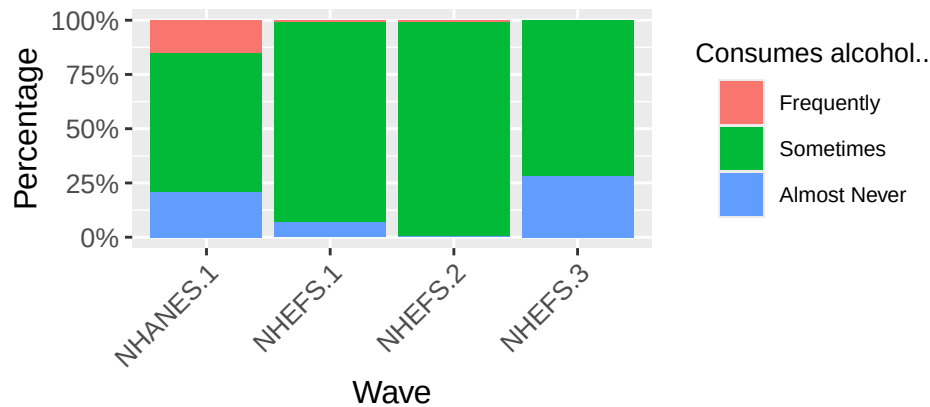
The race composition is highly stable across waves: the cohort is predominantly White, with a small and nearly constant proportion of Black participants and very few classified as Other. This suggests little race-related differential loss to follow-up but also indicates that the cohort is racially homogeneous.

8. Smoke-ratio for the four waves



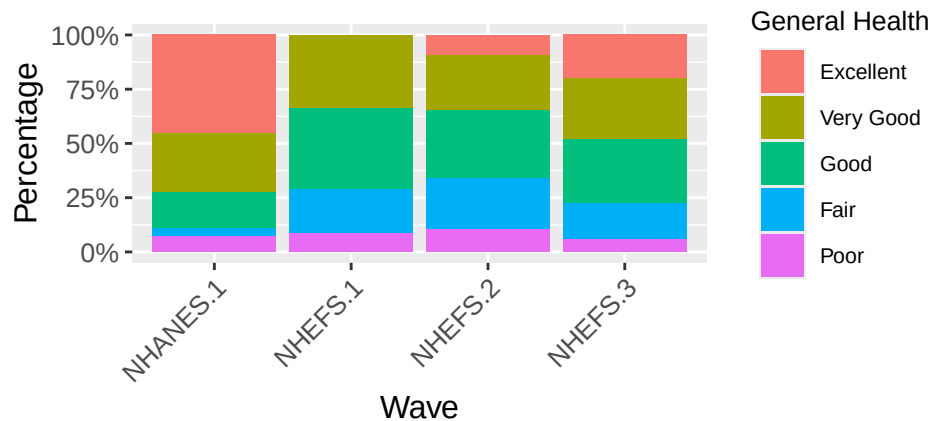
The smoking proportions by wave should be interpreted very cautiously. Because smoking status is missing for most individuals, this plot is based on a relatively small subset of the cohort with non-missing smoke data.

9. Alcohol categories for the four waves



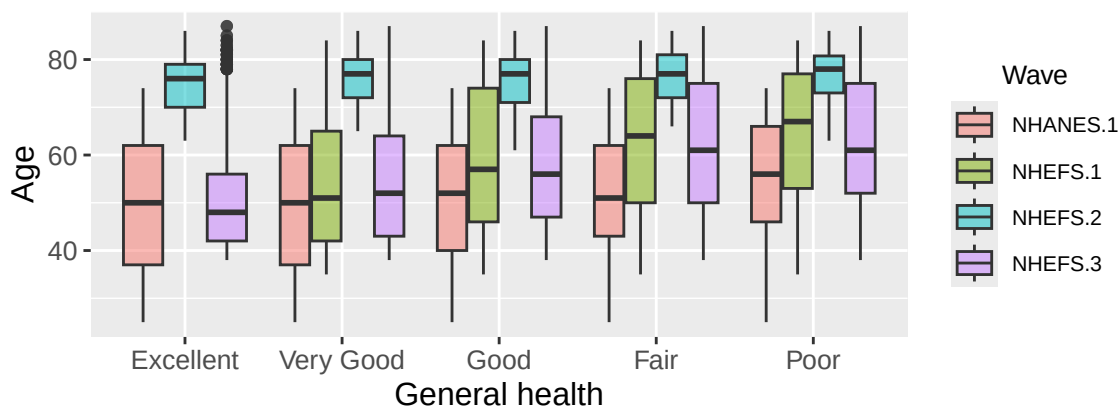
Across all waves, most participants fall in the 'sometimes' drinking category, with relatively few reporting frequent alcohol use. Frequent drinking is most visible at baseline (NHANES.1) and becomes rare in later NHEFS waves, while the proportion reporting 'almost never' drinking increases by NHEFS.3. This suggests a shift toward lower reported alcohol consumption over time.

10. General health for the four waves



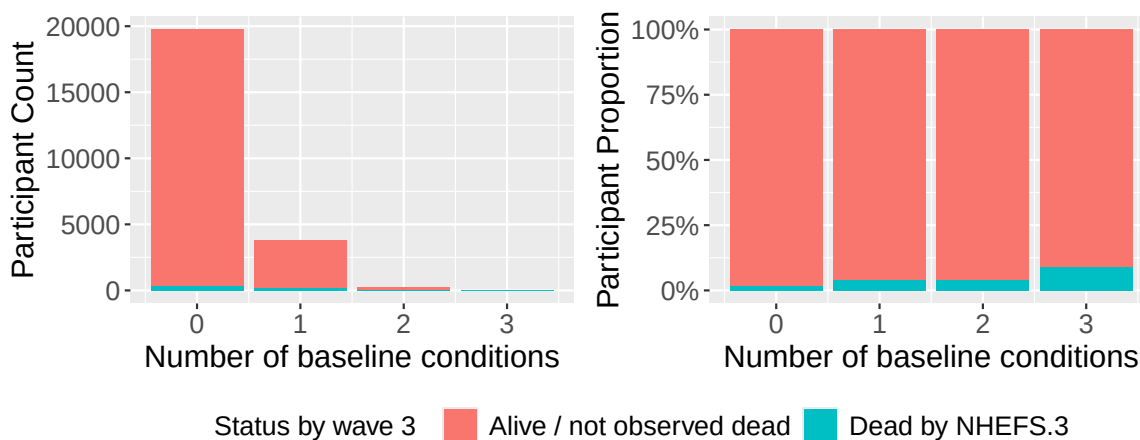
Self-reported general health shifts modestly over time: at baseline a larger share of participants report ‘excellent’ or ‘very good’ health, whereas in later NHEFS waves these top categories shrink and ‘good’ and ‘fair’ become more common, with a small increase in ‘poor’ health as well. Overall, perceived health gradually worsens as the cohort ages, though most individuals continue to rate their health as at least ‘good’.

11. Age distribution by self-reported health and wave



Within each wave, poorer self-reported health is associated with higher age: participants rating their health as ‘fair/poor’ are generally older than those reporting ‘excellent/very good’. Across waves, the age distribution also shifts upward for every health category, reflecting that the cohort ages over time even within the same self-reported health levels.

12. Baseline comorbidity burden and eventual death



The left panel shows that most participants have no baseline conditions, with steadily fewer individuals as comorbidity count increases. The right panel re-expresses this in terms of proportions and highlights the risk gradient: among those with 0 conditions almost everyone is alive/not observed dead by wave 3, while the proportion dead by NHEFS.3 increases clearly with each additional condition and is highest for participants with 3 baseline conditions. Together, the plots show that higher comorbidity is both uncommon and strongly associated with subsequent mortality.

Missingness

```
df.nhf.1 <- data.frame(col = colnames(NHEFS.1),
                      val = round(colMeans(is.na(NHEFS.1)),3))
df.nhf.2 <- data.frame(col = colnames(NHEFS.2),
                      val = round(colMeans(is.na(NHEFS.2)),3))
df.nhf.3 <- data.frame(col = colnames(NHEFS.3),
                      val = round(colMeans(is.na(NHEFS.3)),3))
df.nhn.1 <- data.frame(col = colnames(NHANES.1),
                      val = round(colMeans(is.na(NHANES.1)),3))

df.miss <- list(df.nhf.1, df.nhf.2, df.nhf.3, df.nhn.1) |>
  reduce(full_join, by = "col") |>
  filter(!col %in% c("wave", "ssn"))
df.miss[which(is.na(df.miss), arr.ind = T)] <- 1
colnames(df.miss) <- c("Variable", "NHEFS.1", "NHEFS.2", "NHEFS.3",
                     "NHANES.1")
df.miss <- df.miss |> select(Variable, NHANES.1, everything())

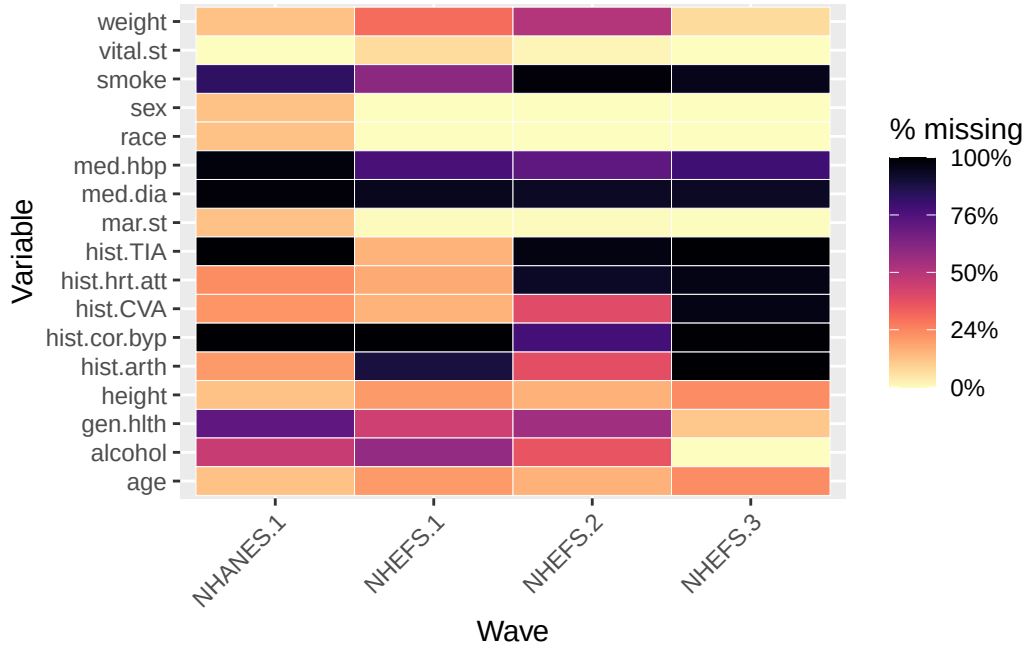
kable(df.miss,
      caption = "Missingness of variabels at each wave\\label{tab:missing}")
```

Table 2: Missingness of variabels at each wave

Variable	NHANES.1	NHEFS.1	NHEFS.2	NHEFS.3
vital.st	0.000	0.071	0.018	0.000
sex	0.128	0.000	0.000	0.000
race	0.128	0.000	0.000	0.000
mar.st	0.128	0.003	0.005	0.002
gen.hlth	0.710	0.443	0.551	0.113
hist.hrt.att	0.238	0.177	0.930	0.965
hist.TIA	1.000	0.159	0.970	1.000

Variable	NHANES.1	NHEFS.1	NHEFS.2	NHEFS.3
hist.CVA	0.222	0.158	0.393	0.962
med.hbp	0.979	0.768	0.715	0.790
med.dia	0.986	0.945	0.934	0.931
smoke	0.829	0.604	0.983	0.954
alcohol	0.457	0.587	0.364	0.000
hist.arth	0.210	0.887	0.386	1.000
weight	0.128	0.308	0.506	0.072
height	0.128	0.212	0.163	0.240
age	0.128	0.212	0.163	0.240
hist.cor.byp	1.000	1.000	0.780	0.995

Missingness plot: We present the above table visually through the following plot.



Missing data patterns and implications

Table 2 shows substantial heterogeneity in the amount of missing data across variables and waves. Identifiers and basic demographics such as `ssn`, `sex`, `race`, and `mar.st` are essentially complete in the NHEFS waves (missingness close to 0%), with only modest missingness at baseline for `race` and `mar.st` (about 13%). In contrast, several clinical and behavioural variables exhibit very high levels of missingness. For example, medication indicators `med.hbp`

and `med.dia` are missing for roughly 70–80% of subjects in the NHEFS waves and for nearly all subjects at baseline; the coronary bypass indicator `hist.cor.byp` is missing for about 80% of subjects in NHEFS.2 and essentially 100% in the other waves. Smoking and alcohol variables also suffer from considerable missingness: `smoke` is missing for roughly 60% of participants in NHEFS.1 and almost all participants in NHEFS.2 and NHEFS.3, and `alcohol` is missing for about 40–60% of subjects at most waves. Measures like `weight`, `height`, and derived `age` have moderate missingness (roughly 15–50%, depending on wave), while several history variables (`hist.hrt.att`, `hist.TIA`, `hist.CVA`, `hist.arth`) show a mixture of moderate (15–40%) and extreme (> 90%) missingness across waves. General health (`gen.hlth`) is observed for only about half of the subjects in the NHEFS waves and not collected at all at baseline.

These patterns indicate that missingness is driven by a combination of study design (e.g., detailed examinations or questions asked only in subsamples or at particular waves) and subject characteristics, rather than being Missing Completely At Random (MCAR). In particular, the very high and correlated missingness for baseline clinical and behavioural variables such as `med.hbp`, `med.dia`, `smoke`, `alcohol`, and disease history measures suggests that baseline *exam status* and underlying health are predictive of whether related data are observed in the NHEFS waves. Conversely, basic demographic variables (`sex`, `race`, `mar.st`) and identifiers (`ssn`) are essentially complete and therefore unlikely to drive missingness patterns.

Implications for subsequent analysis.

- Because missingness is strongly related to baseline health and to which questionnaire/exam modules were administered, complete-case analyses would disproportionately retain a healthier and more thoroughly examined subset of participants. This would bias estimates of risk factor distributions and could distort any clustering of trajectories by under-representing high-risk or less compliant individuals.
- Several variables (notably `med.dia`, `med.hbp`, `hist.cor.byp`, and smoking history in later waves) are so sparsely observed that they are effectively unusable for longitudinal clustering unless one adopts a model that explicitly accounts for the missingness mechanism. In practice, these variables may need to be excluded from the clustering feature set or handled via multiple imputation or joint modelling.
- Moderate missingness in key continuous variables such as `weight`, `height`, and `age`, together with differential loss to follow-up and mortality (summarised by `vital.st`), implies a risk of survivorship bias: subjects with worse baseline health are more likely to drop out or die, so observed follow-up data over-represent healthier trajectories. Any clustering of longitudinal profiles should therefore acknowledge that the observed data arise from a selected subset of the original cohort.

Overall, the updated missingness table reinforces that the data are unlikely to be MCAR. Baseline health and examination variables are closely tied to the availability of follow-up measurements, and subsequent clustering or trajectory analyses must handle missing data explicitly rather than relying on naive complete-case methods.

Distribution of *ssn* in each data sets.

The following code generates a matrix where the diagonal elements show how many participants are in that particular data set. And for any (i, j) -th non-diagonal element, it gives the number of participants from i -th data set that is present on the j -th data set.

```
dfs <- list( NHANES.1 = NHANES.1, NHEFS.1 = NHEFS.1,
            NHEFS.2 = NHEFS.2, NHEFS.3 = NHEFS.3)

# Unique ssn sets for each data set
# ssn_list <- purrr::map(dfs, ~ base::unique(.x$ssn))
# save(ssn_list, file = "ssn_list.Rdata")
load("ssn_list.Rdata")

overlap_count <- function(i, j) {
  sum(ssn_list[[i]] %in% ssn_list[[j]])
}

n_sets <- length(ssn_list)
overlap_mat <- matrix(
  nrow = n_sets, ncol = n_sets,
  dimnames = list(names(ssn_list), names(ssn_list))
)

for (i in seq_len(n_sets)) {
  for (j in seq_len(n_sets)) {
    overlap_mat[i, j] <- overlap_count(i, j)
  }
}

overlap_mat
```

	NHANES.1	NHEFS.1	NHEFS.2	NHEFS.3
NHANES.1	23808	14407	5677	9998
NHEFS.1	14407	14407	5677	9998
NHEFS.2	5677	5677	5677	2894
NHEFS.3	9998	9998	2894	9998

We see that the NHANES.1 data set is the super set and all the participants from the subsequent NHEFS waves are in NHANES.1. The same goes for NHEFS.1 data set. It contains all participants from following NHEFS data sets.

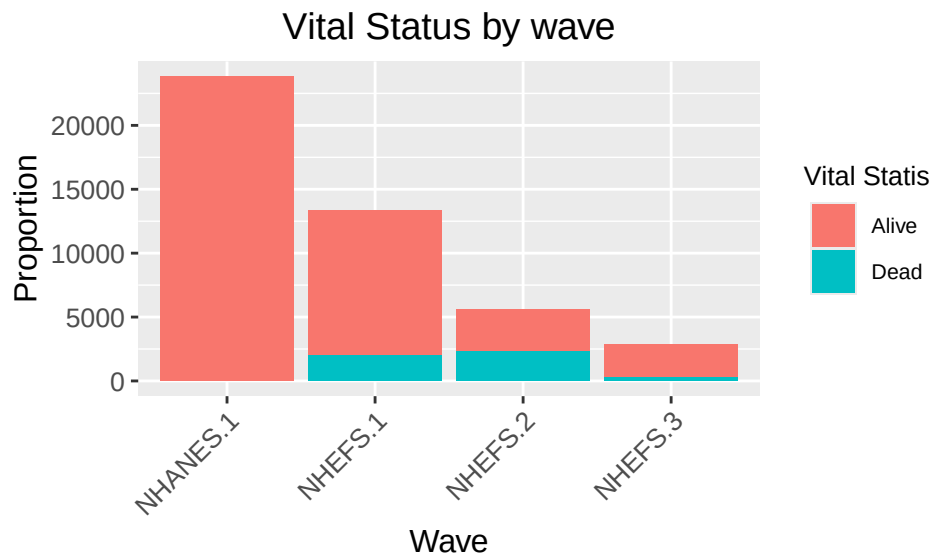
Things change for NHEFS.2 data set. We see that among 5677 participants only 2894 participants are in NHEFS.3 and there are $(9998-2894) = 7104$ extra participant that are not in NHEFS.02. This is a common phenomena in longitudinal study which can be attributed to funding availability or participants willingness to take part again, or maybe mere designing.

In summary, we see the only 2894 participants are common in the four data sets, which is respectively 0.1215558, 0.2008746, 0.5097763, and 0.2894579 proportions of data for the NHANES, NHEFS 1, NHEFS 2, and NHEFS 3 data set.

Below we subset the NHEFS.03 data set to all the common participants who were also in NHEFS.02 data set to make things easier and easy to interpret.

```
ssn_keep <- intersect(unique(NHEFS.2$ssn), unique(NHEFS.3$ssn))  
  
NHEFS.3.com <- NHEFS.3 |> filter(ssn %in% ssn_keep)
```

Vital status for the four waves



The plot shows how vital status evolves across waves. At baseline (NHANES.1) all participants are alive (by design), whereas in the follow-up NHEFS waves an increasing number are recorded as dead, reflecting cumulative mortality over time.

Clustering NHANES data

We start with subsetting the NHANES data set. We choose complete case rows and also we choose variables which will keep the number of participants within a reasonable range.

```
# Choosing variables for clustering the NHANES I data set

vars_cont <- c("age", "height", "weight")
vars_cat  <- c("sex", "race", "mar.st", "gen.hlth", "alcohol",
              "hist.hrt.att", "hist.CVA", "hist.arth")

NHANES.1.sub <- NHANES.1 |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  filter(complete.cases(across(-ssn))) |>
  mutate(across(any_of(vars_cat), as.factor))

dim(NHANES.1.sub)
```

```
[1] 2597    12
```

We see that there are altogether 2597 participants with complete cases for the variables chosen above. We fit the four methods one by one on this subset of NHANES data set.

There could have been a better way of handling the missing data but we couldn't explore due to the lack of time.

Fiting the k-means clusering

```
# selecting variables with only 1 level
one_level_vars <- NHANES.1.sub |>
  select(-ssn) |>
  summarise(across(everything(), n_distinct)) |>
  pivot_longer(cols = everything(),
               names_to = "variable", values_to = "n_levels") |>
  filter(n_levels <= 1) |>
  pull(variable)

nhanes_km <- NHANES.1.sub |>
  select(-all_of(one_level_vars))
```

```

# creating the design matrix and scaling
X_base <- model.matrix(~ . - ssn - 1, data = nhanes_km)
X_base_scaled <- scale(X_base)

# Running k-means for several K
K_grid <- 2:15

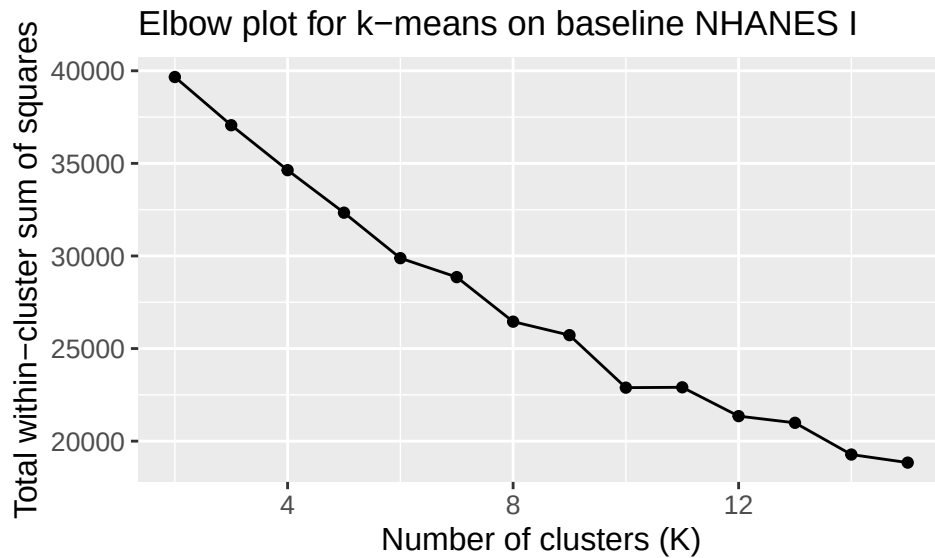
set.seed(50)
seeds <- sample.int(1e7, length(K_grid))

# km_list <- mclapply(
#   seq_along(K_grid),
#   function(i) {
#     set.seed(seeds[i])
#     kmeans(X_base_scaled, centers = K_grid[i], nstart = 50)
#   },
#   mc.cores = detectCores(),
#   mc.set.seed = FALSE
# )
#
# save(km_list, file = "fit.km.0.Rdata")

load("fit.km.0.Rdata")

# Plotting the elbow plot
data.frame(
  K = K_grid,
  WSS = sapply(km_list, function(fit) fit$tot.withinss)
) |>
  ggplot(aes(x = K, y = WSS)) +
  geom_line() +
  geom_point() +
  labs(x = "Number of clusters (K)",
       y = "Total within-cluster sum of squares",
       title = "Elbow plot for k-means on baseline NHANES I") +
  theme(axis.text = element_text(size = 10),
        axis.title = element_text(size = 12))

```



Even with the `set.seed()` option, we get different results for different run. But based on multiple run of the code, $K = 6$ seems to be a good choice.

```
# Choosing K and fit final k-means

K_choose <- 6

set.seed(2025)
km_final <- kmeans(X_base_scaled, centers = K_choose, nstart = 100)

NHANES.1.sub <- NHANES.1.sub |>
  mutate(cluster_km = factor(km_final$cluster))

# cluster sizes
table(NHANES.1.sub$cluster_km)
```

1	2	3	4	5	6
23	92	153	1185	170	974

Fiting the Bayesian Gaussian Mixture Model

To cluster the baseline NHANES I subjects we treat each participant $i = 1, \dots, N$ as having a D -dimensional vector of standardized features $\mathbf{x}_i = (x_{i1}, \dots, x_{iD})^\top$ constructed from age, height, weight, and selected categorical variables (after dummy coding and centering / scaling). We assume that the population is a finite mixture of K latent subpopulations (clusters). Let $z_i \in \{1, \dots, K\}$ denote the latent cluster allocation for subject i , and let $\theta = (\theta_1, \dots, \theta_K)^\top$ be the corresponding vector of mixing proportions.

Latent allocation and observation model.

Conditional on the mixture weights θ , the cluster indicator for subject i follows a categorical distribution,

$$z_i \mid \theta \sim \text{Categorical}(\theta), \quad i = 1, \dots, N.$$

Given $z_i = k$, the feature vector $\mathbf{x}_i$ is assumed to arise from a D -dimensional Gaussian distribution with cluster-specific mean $\mu_k = (\mu_{k1}, \dots, \mu_{kD})^\top$ and diagonal covariance matrix $\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kD}^2)$:

$$\mathbf{x}_i \mid z_i = k, \mu_k, \sigma_k \sim \mathcal{N}_D(\mu_k, \Sigma_k), \quad k = 1, \dots, K,$$

where $\sigma_k = (\sigma_{k1}, \dots, \sigma_{kD})^\top$ collects the cluster-specific standard deviations.

Marginalising over z_i yields the usual finite mixture representation,

$$p(\mathbf{x}_i \mid \theta, \{\mu_k, \sigma_k\}_{k=1}^K) = \sum_{k=1}^K \theta_k \mathcal{N}_D(\mathbf{x}_i \mid \mu_k, \Sigma_k).$$

Prior distributions.

We place weakly informative priors on the mixing weights, cluster means, and cluster-specific standard deviations. In particular,

$$\theta \sim \text{Dirichlet}(\alpha_1, \dots, \alpha_K), \quad \text{with } \alpha_k = 1 \text{ for all } k,$$

so that a priori all cluster sizes are equally likely. For the Gaussian parameters we use independent priors across clusters and dimensions,

$$\mu_{kd} \sim \mathcal{N}(0, \tau^2), \quad \sigma_{kd} \sim \text{Lognormal}(0, \sigma_\ell^2), \quad k = 1, \dots, K, \quad d = 1, \dots, D,$$

with τ^2 and σ_ℓ^2 chosen to be moderately large (in our implementation we take $\tau = 2$ and $\sigma_\ell = 0.5$) so that the priors are weakly informative on the standardized scale.

We encode the above model in the following Stan code.

```

options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

bgmm_stan_code <- "
data {
  int<lower=1> N;           // number of subjects
  int<lower=1> D;           // number of features
  int<lower=1> K;           // number of clusters
  matrix[N, D] X;         // scaled features
}
parameters {
  simplex[K] theta;        // mixing proportions
  matrix[K, D] mu;         // cluster means
  matrix<lower=0>[K, D] sigma; // cluster-specific SDs (diagonal covariances)
}
model {
  // priors
  theta ~ dirichlet(rep_vector(1.0, K)); // vague Dirichlet
  to_vector(mu) ~ normal(0, 2);           // weakly informative
  to_vector(sigma) ~ lognormal(0, 0.5);    // positive SDs

  // likelihood (marginal over latent labels)
  for (n in 1:N) {
    vector[K] lps;
    for (k in 1:K) {
      lps[k] = log(theta[k]);
      for (d in 1:D) {
        lps[k] += normal_lpdf(X[n, d] | mu[k, d], sigma[k, d]);
      }
    }
    target += log_sum_exp(lps);
  }
}
"

```

We fit the model with the same scaled data matrix we created while fitting the k-means clustering.

```

N <- nrow(X_base_scaled)
D <- ncol(X_base_scaled)
K <- K_choose

```

```

bgmm_data <- list(N = N, D = D, K = K, X = X_base_scaled)

# Fitting the Bayesian GMM via Stan

# set.seed(100)
# bgmm_fit.0 <- stan(
#   model_code = bgmm_stan_code,
#   data       = bgmm_data,
#   iter       = 2000,
#   warmup     = 1000,
#   chains     = 1,
#   cores      = detectCores(),
#   seed       = 100
# )
#
# # computing posterior means for theta, mu, sigma
# theta_hat <- colMeans(as.data.frame(extract(bgmm_fit.0, "theta")$theta))
# mu_hat    <- apply(extract(bgmm_fit.0, "mu")$mu,      c(2, 3), mean)
# sigma_hat <- apply(extract(bgmm_fit.0, "sigma")$sigma, c(2, 3), mean)
#
# bgmm_list.0 <- list(theta_hat, mu_hat, sigma_hat)
#
# save(bgmm_list.0, file = "fit.bgmm.0.Rdata")

load("fit.bgmm.0.Rdata")

```

Now we obtain the cluster assignments with the following code.

```

# computing posterior means for theta, mu, sigma
theta_hat <- bgmm_list.0[[1]]
mu_hat    <- bgmm_list.0[[2]]
sigma_hat <- bgmm_list.0[[3]]

# computing posterior cluster probabilities for each individual
log_resp <- matrix(NA_real_, nrow = N, ncol = K)

for (n in 1:N) {
  for (k in 1:K) {
    lp <- log(theta_hat[k])
    for (d in 1:D) {
      lp <- lp + dnorm(X_base_scaled[n, d],
                      mean = mu_hat[k, d],

```

```

        sd    = sigma_hat[k, d],
        log    = TRUE)
    }
    log_resp[n, k] <- lp
  }
}

# normalizing to probabilities
resp <- exp(log_resp - apply(log_resp, 1, max))
resp <- resp / rowSums(resp)

# MAP cluster labels
cluster_bgmm <- max.col(resp)

NHANES.1.sub$cluster_bgmm <- factor(cluster_bgmm)

# cluster sizes
table(NHANES.1.sub$cluster_bgmm)

```

1	2	3	4
1434	2	19	1142

Fiting the CLAMR Model

To incorporate clinical knowledge about plausible ranges of baseline variables, we also fit the CLAMR (Clinically informed Latent Allocation Mixture Regression) model in the spirit of Dombowsky et al. (2025) to the baseline NHANES I data. As before, each subject $i = 1, \dots, N$ has a D -dimensional feature vector $\mathbf{x}_i = (x_{i1}, \dots, x_{iD})^\top$, but here we restrict attention to continuous baseline variables such as age, height, and weight for which it is meaningful to define clinically interpretable ranges.

We assume a finite mixture of K latent clusters with mixing proportions $\theta = (\theta_1, \dots, \theta_K)^\top$ and cluster-specific means $\mu_k = (\mu_{k1}, \dots, \mu_{kD})^\top$ and diagonal covariance matrices $\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kD}^2)$.

Meaningful regions for each feature.

For each continuous feature $d \in \{1, \dots, D\}$ we define L *meaningful regions* (MRs) on the real line,

$$\mathcal{R}_{d1}, \dots, \mathcal{R}_{dL} \subset \mathbb{R},$$

intended to correspond to clinically interpretable ranges such as “low”, “normal”, and “high” values. In our implementation we take $L = 3$ and construct these regions from the empirical tertiles of the baseline distribution of $x_{\cdot d}$:

$$\mathcal{R}_{d1} = [q_{d,0}, q_{d,1/3}], \quad \mathcal{R}_{d2} = (q_{d,1/3}, q_{d,2/3}], \quad \mathcal{R}_{d3} = (q_{d,2/3}, q_{d,1}],$$

where $q_{d,p}$ denotes the p th empirical quantile of $x_{\cdot d}$ and $q_{d,0}, q_{d,1}$ are the minimum and maximum. For each region $\mathcal{R}_{d\ell} = [a_{d\ell}, b_{d\ell}]$ we define a midpoint $m_{d\ell} = (a_{d\ell} + b_{d\ell})/2$ and a kernel standard deviation $\tau_{d\ell}$ chosen so that a Normal distribution with mean $m_{d\ell}$ and standard deviation $\tau_{d\ell}$ places most of its mass inside the interval; in practice we take

$$\tau_{d\ell} \approx \frac{b_{d\ell} - a_{d\ell}}{2z_{0.975}},$$

so that roughly 95% of the mass lies in $[a_{d\ell}, b_{d\ell}]$.

Latent allocation and likelihood.

Conditional on the mixing proportions θ , the cluster indicator for subject i is

$$z_i \mid \theta \sim \text{Categorical}(\theta), \quad i = 1, \dots, N.$$

Given $z_i = k$ and the cluster-specific parameters, the continuous feature vector obeys a diagonal-covariance Gaussian model

$$\mathbf{x}_i \mid z_i = k, \mu_k, \Sigma_k \sim \mathcal{N}_D(\mu_k, \Sigma_k),$$

where $\sigma_k = (\sigma_{k1}, \dots, \sigma_{kD})^\top$ and $\Sigma_k = \text{diag}(\sigma_{k1}^2, \dots, \sigma_{kD}^2)$. Marginalising over z_i again yields the finite mixture density

$$p(\mathbf{x}_i \mid \theta, \{\mu_k, \sigma_k\}_{k=1}^K) = \sum_{k=1}^K \theta_k \mathcal{N}_D(\mathbf{x}_i \mid \mu_k, \Sigma_k).$$

CLAMR prior on cluster centers.

The key difference between CLAMR and the standard Gaussian mixture model lies in the prior for the cluster centers μ_{kd} . For each feature d we introduce a simplex $\pi_d = (\pi_{d1}, \dots, \pi_{dL})^\top$ of probabilities over its meaningful regions. We assign a Dirichlet prior,

$$\pi_d \sim \text{Dirichlet}(\alpha_{d1}, \dots, \alpha_{dL}), \quad d = 1, \dots, D,$$

with $\alpha_{d\ell} = 1$ in our implementation to represent a priori indifference among regions.

Conditional on π_d , the prior for each cluster center coordinate μ_{kd} is a mixture of Gaussian kernels centred at the MR midpoints,

$$p(\mu_{kd} \mid \pi_d) = \sum_{\ell=1}^L \pi_{d\ell} \mathcal{N}(\mu_{kd} \mid m_{d\ell}, \tau_{d\ell}^2), \quad k = 1, \dots, K, \quad d = 1, \dots, D.$$

This construction encourages each cluster mean to lie near one of the clinically meaningful regions for that feature, while still allowing the data to update both the location and the relative weight of the regions through π_d .

For the remaining parameters we use weakly informative priors similar to the standard Bayesian Gaussian mixture:

$$\theta \sim \text{Dirichlet}(1, \dots, 1), \quad \sigma_{kd} \sim \text{Lognormal}(0, 0.5^2), \quad k = 1, \dots, K, \quad d = 1, \dots, D.$$

We encode the above model in the following Stan code.

```
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

clamr_stan_code <- "
data {
  int<lower=1> N;           // subjects
  int<lower=1> D;           // features
  int<lower=1> K;           // overfitted # of clusters
  int<lower=1> L;           // # meaningful regions per feature

  matrix[N, D] X;          // data (original scale)
```

```

// CLAMR prior hyperparameters:
matrix[D, L] mr_mid;           // midpoints of MRs for each feature
matrix[D, L] mr_sd;           // SDs for each MR kernel
matrix[D, L] alpha_mr;        // Dirichlet params for MR probs
}
parameters {
  simplex[K] theta;            // mixing proportions

  matrix[K, D] mu;             // cluster centers
  matrix<lower=0>[K, D] sigma; // cluster SDs (diagonal covariance)

  // MR probabilities per feature (each row is a simplex over L regions)
  simplex[L] pi_mr[D];
}
model {
  // Priors on mixture weights and variances
  theta ~ dirichlet(rep_vector(1.0, K));
  to_vector(sigma) ~ lognormal(0, 0.5);

  // Prior on MR probabilities for each feature
  for (d in 1:D) {
    pi_mr[d] ~ dirichlet( to_vector(alpha_mr[d]) );
  }

  // CLAMR prior on cluster centers: mixture of Gaussian kernels over MRs
  for (k in 1:K) {
    for (d in 1:D) {
      vector[L] lp_comp;
      for (l in 1:L) {
        lp_comp[l] = log(pi_mr[d][l]) +
                      normal_lpdf(mu[k, d] | mr_mid[d, l], mr_sd[d, l]);
      }
      target += log_sum_exp(lp_comp);
    }
  }

  // Likelihood: diagonal Gaussian mixture
  for (n in 1:N) {
    vector[K] lp_cluster;
    for (k in 1:K) {
      lp_cluster[k] = log(theta[k]);
      for (d in 1:D) {

```

```

        lp_cluster[k] += normal_lpdf(X[n, d] | mu[k, d], sigma[k, d]);
      }
    }
    target += log_sum_exp(lp_cluster);
  }
}
"

```

Now we prepare the dataset required for fitting the model. We take the continuous variables and define meaningful regions for each of them. We take $L = 3$ for each of the continuous variable, denoting high, normal and low regions.

```

mr_vars <- vars_cont

# subset to the continuous variables
nhanes_clamr <- NHANES.1.sub |>
  select(ssn, all_of(mr_vars))

L <- 3

# computing MR boundaries using empirical tertiles
mr_bounds <- lapply(mr_vars, function(v) {
  x <- nhanes_clamr[[v]]
  qs <- quantile(x, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE)
  as.numeric(qs)
})
names(mr_bounds) <- mr_vars

# midpoints and within-region SDs for CLAMR prior
# we choose sigma so that about 95% of a Normal lies inside each interval
z975 <- qnorm(0.975)

mr_mid <- matrix(NA_real_, nrow = length(mr_vars), ncol = L)
mr_sd <- matrix(NA_real_, nrow = length(mr_vars), ncol = L)

for (j in seq_along(mr_vars)) {
  b <- mr_bounds[[j]]
  for (l in 1:L) {
    a_l <- b[l]
    b_l <- b[l + 1]
    mr_mid[j, l] <- (a_l + b_l) / 2
    mr_sd[j, l] <- (b_l - a_l) / (2 * z975) # so ~95% lies in [a_l, b_l]
  }
}

```



```

    }
  }

# simpling symmetric Dirichlet hyperparameters over MRs
alpha_mr <- matrix(1, nrow = length(mr_vars), ncol = L)

# building the data matrix for modeling
X_clamr <- as.matrix(nhanes_clamr[, mr_vars])

N <- nrow(X_clamr)
D <- ncol(X_clamr)
K <- K_choose

clamr_data <- list(
  N      = N,
  D      = D,
  K      = K,
  L      = L,
  X      = X_clamr,
  mr_mid = mr_mid,
  mr_sd  = mr_sd,
  alpha_mr = alpha_mr
)

```

We fit the model as follows.

```

# set.seed(100)
# clamr_fit.0 <- stan(
#   model_code = clamr_stan_code,
#   data       = clamr_data,
#   iter       = 2000,
#   warmup     = 1000,
#   chains     = 1,
#   cores      = detectCores(),
#   seed       = 100
# )
#
# post <- extract(clamr_fit.0)
# theta_hat <- colMeans(post$theta)
# mu_hat    <- apply(post$mu,    c(2, 3), mean)
# sigma_hat <- apply(post$sigma, c(2, 3), mean)
#

```

```
# clamr_list.0 <- list(theta_hat, mu_hat, sigma_hat)
#
# save(clamr_list.0, file = "fit.clamr.0.Rdata")

load("fit.clamr.0.Rdata")
```

And subsequently assign the clusters as follows.

```
theta_hat <- clamr_list.0[[1]]
mu_hat     <- clamr_list.0[[2]]
sigma_hat  <- clamr_list.0[[3]]

log_resp <- matrix(NA_real_, nrow = N, ncol = K)

for (n in 1:N) {
  for (k in 1:K) {
    lp <- log(theta_hat[k])
    for (d in 1:D) {
      lp <- lp + dnorm(X_clamr[n, d],
                      mean = mu_hat[k, d],
                      sd   = sigma_hat[k, d],
                      log   = TRUE)
    }
    log_resp[n, k] <- lp
  }
}

resp <- exp(log_resp - apply(log_resp, 1, max))
resp <- resp / rowSums(resp)

cluster_clamr <- max.col(resp) # MAP cluster

NHANES.1.sub$cluster_clamr <- factor(cluster_clamr)
table(NHANES.1.sub$cluster_clamr)
```

```
  1   2   3   4   5   6
601 338 126 959 355 218
```

Fitting the Bayesian pyramid method

To flexibly model the joint distribution of multivariate categorical baseline variables, we also consider the Bayesian Pyramids framework of Gu and Dunson (2023), which introduces a multilayer discrete latent structure specifically tailored to high-dimensional unordered categorical data. Let

$$\mathbf{y}_i = (y_{i1}, \dots, y_{ip})^\top, \quad i = 1, \dots, n,$$

denote the p observed categorical variables for subject i , with $y_{ij} \in \{1, \dots, d_j\}$ and d_j the number of categories for variable j .

Multilayer latent structure.

Bayesian Pyramids posit several layers of binary latent variables arranged in a “pyramid” shape. Let D denote the number of latent layers (the depth of the pyramid). The layer closest to the observed data is

$$\alpha_i^{(1)} = (\alpha_{i1}^{(1)}, \dots, \alpha_{iK_1}^{(1)})^\top \in \{0, 1\}^{K_1},$$

and for $m = 1, \dots, D - 1$ we introduce deeper layers

$$\alpha_i^{(m+1)} \in \{0, 1\}^{K_{m+1}},$$

with $K_{m+1} \leq K_m$ so that the number of latent variables decreases as we move deeper in the pyramid. At the very top of the hierarchy, there is a single discrete latent variable

$$z_i = z_i^{(D)} \in \{1, \dots, B\},$$

which can be interpreted as a coarse high-level cluster or latent class. The complete latent architecture therefore consists of D latent layers plus the observed layer at the bottom.

Pyramid-shaped Bayesian network and factorization.

The latent and observed variables are linked via a directed acyclic graph (DAG) with edges only between adjacent layers and directed from deep to shallow (top-down). There are no edges within a layer. Writing $\alpha_i^{(m)} = (\alpha_{i1}^{(m)}, \dots, \alpha_{iK_m}^{(m)})$ for layer m and suppressing the dependence on i for notational simplicity, the joint distribution of $\mathbf{y}_i$, the latent layers $\{\alpha_i^{(m)}\}_{m=1}^{D-1}$, and the top latent z_i admits the factorization

$$P(\mathbf{y}_i, \{\alpha_i^{(m)}\}_{m=1}^{D-1}, z_i) = P(\mathbf{y}_i \mid \alpha_i^{(1)}) \prod_{m=1}^{D-2} P(\alpha_i^{(m)} \mid \alpha_i^{(m+1)}) P(\alpha_i^{(D-1)} \mid z_i) P(z_i). \quad (1)$$

The conditional distribution of $\mathbf{y}_i$ given the closest latent layer $\alpha_i^{(1)}$ factorizes across coordinates,

$$P(\mathbf{y}_i \mid \alpha_i^{(1)}) = \prod_{j=1}^p P(y_{ij} \mid \alpha_i^{(1)}),$$

with each $P(y_{ij} \mid \alpha_i^{(1)})$ modelled using a multinomial logistic regression on a subset of the binary latent features in $\alpha_i^{(1)}$. Analogously, each deeper layer $\alpha_i^{(m)}$ is modelled conditionally on the next deeper layer $\alpha_i^{(m+1)}$ —again via logistic regressions—with sparse parent sets encoded by binary graphical matrices. This pyramid-shaped Bayesian network induces rich dependence among the observed variables while maintaining parsimony and interpretability.

Two-latent-layer Bayesian Pyramid and inference.

For applied work the authors focus on a two-latent-layer model, which we also adopt in our implementation. In this case, each subject has:

- a top latent class $z_i \in \{1, \dots, B\}$, representing a coarse discrete cluster; and
- a vector of binary latent features $\alpha_i^{(1)} \in \{0, 1\}^{K_1}$, representing more granular latent traits.

The conditional distributions $P(\alpha_i^{(1)} \mid z_i)$ and $P(y_{ij} \mid \alpha_i^{(1)})$ are specified via logistic and multinomial-logit regressions with sparse connectivity, and a shrinkage prior is placed on the regression coefficients and graphical connections to favour parsimonious latent structure. Posterior computation proceeds via a tailored Gibbs sampler using Pólya–Gamma data augmentation for the logistic components, which allows efficient updating of both the latent binary vectors and the regression parameters.

We encode the above model in the following Stan code.

```
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)

pyramid_stan_code <- "
data {
  int<lower=1> N;                // number of subjects
  int<lower=1> P;                // number of variables
  int<lower=1> K;                // number of clusters
  int<lower=1> C[P];             // number of categories per variable
  int<lower=1> Y[N, P];          // categorical responses (1..C[p])
}
parameters {
  simplex[K] theta;              // mixing proportions
  // phi[k, p] is a simplex over categories 1..C[p]
  simplex[C[p]] phi[K, P];       // cluster- and variable-specific probs
}
model {
  // priors
```

```

theta ~ dirichlet(rep_vector(1.0, K));      // vague over cluster sizes
for (k in 1:K) {
  for (p in 1:P) {
    phi[k, p] ~ dirichlet(rep_vector(1.0, C[p])); // uniform over categories
  }
}

// likelihood, marginalizing over z_i
for (n in 1:N) {
  vector[K] lp_k;
  for (k in 1:K) {
    lp_k[k] = log(theta[k]);
    for (p in 1:P) {
      lp_k[k] += categorical_lpmf(Y[n, p] | phi[k, p]);
    }
  }
  target += log_sum_exp(lp_k);
}
}
"

```

Similar as before, we now prepare the datasets.

```

vars_pyr <- vars_cat

nhanes_pyr <- NHANES.1.sub |> select(ssn, any_of(vars_pyr)) |>
  mutate(across(all_of(vars_pyr), ~ as.integer(factor(.))))

# integer matrix of categories
Y_cat <- as.matrix(nhanes_pyr[, vars_pyr])

N <- nrow(Y_cat)
P <- ncol(Y_cat)

# number of categories per variable j
C <- apply(Y_cat, 2, max)
C_max <- max(C)
K_pyr <- K_choose

pyr_data <- list(
  N      = N,
  P      = P,

```

```

K      = K_pyr,
C_max  = C_max,
Y      = Y_cat
)

```

We fit the model as follows.

```

# set.seed(100)
# pyr_fit.0 <- stan(
#   model_code = pyramid_stan_code,
#   data       = pyr_data,
#   iter       = 2000,
#   warmup     = 1000,
#   chains     = 1,
#   cores      = detectCores(),
#   seed       = 100
# )
#
# post <- extract(pyr_fit.0)
#
# theta_hat <- colMeans(post$theta)
# phi_array <- post$phi
# phi_hat <- apply(phi_array, c(2, 3, 4), mean)
#
# pyr_list.0 <- list(theta_hat, phi_hat)
#
# save(pyr_list.0, file = "fit.pyr.0.Rdata")

load("fit.pyr.0.Rdata")

```

And we assign the cluster with the following code.

```

theta_hat <- pyr_list.0[[1]]
phi_hat <- pyr_list.0[[2]]

# computing log posterior responsibilities given a
#           log p(z_i = k | Y_i, theta_hat, phi_hat)
log_resp_pyr <- matrix(NA_real_, nrow = N, ncol = K_pyr)

for (n in 1:N) {
  for (k in 1:K_pyr) {

```

```

    lp <- log(theta_hat[k])
    for (p in 1:P) {
      cat_idx <- Y_cat[n, p]      # in 1..C_max
      lp <- lp + log(phi_hat[k, p, cat_idx])
    }
    log_resp_pyr[n, k] <- lp
  }
}

# normalizing to get posterior probs
resp_pyr <- exp(log_resp_pyr - apply(log_resp_pyr, 1, max))
resp_pyr <- resp_pyr / rowSums(resp_pyr)

# MAP cluster assignment
cluster_pyr <- max.col(resp_pyr)

NHANES.1.sub$cluster_pyr <- factor(cluster_pyr)

# cluster sizes
table(NHANES.1.sub$cluster_pyr)

```

1	2	3	4	5	6
9	5	6	35	2521	21

Finding similarities between clustering procedures

We calculate the similarities between the clusters by calculating the Adjusted Rand Index (ARI) using the `adjustedRandIndex()` function from the `hclust` package.

```
df_clust <- NHANES.1.sub
```

```
# Calculating ARIs
```

```
(ari_km_bgmm <- adjustedRandIndex(df_clust$cluster_km,  
                                  df_clust$cluster_bgmm))
```

```
[1] 0.1094763
```

```
(ari_km_clamr <- adjustedRandIndex(df_clust$cluster_km,  
                                   df_clust$cluster_clamr))
```

```
[1] 0.1383834
```

```
(ari_km_pyr <- adjustedRandIndex(df_clust$cluster_km,  
                                 df_clust$cluster_pyr))
```

```
[1] 0.05794397
```

```
(ari_bgmm_clamr <- adjustedRandIndex(df_clust$cluster_bgmm,  
                                      df_clust$cluster_clamr))
```

```
[1] 0.04284857
```

```
(ari_bgmm_pyr <- adjustedRandIndex(df_clust$cluster_bgmm,  
                                   df_clust$cluster_pyr))
```

```
[1] 0.02710135
```

```
(ari_clamr_pyr <- adjustedRandIndex(df_clust$cluster_clamr,  
                                    df_clust$cluster_pyr))
```

```
[1] 0.005180211
```

We see that the results are **not at all good**.

Implementation and practical challenges

For K -means we first dummy-coded categorical variables and standardized all features to have a mean of zero and unit variance. Because the algorithm cannot handle missing values, we restricted attention to subjects with complete data on the selected baseline covariates, resulting in $n = 2597$ individuals. The number of clusters was chosen as $K = 6$ based on an elbow plot of the within-cluster sum of squares and interpretability of the resulting clusters.

The BGMM used the same standardized design matrix and assumed a finite mixture of K multivariate Gaussians with diagonal covariance matrices and weakly informative priors on the mixture weights, means, and standard deviations. Implementing the BGMM was computationally demanding with $n = 2597$ complete cases, $D = 18$ features, and $K = 6$ clusters, Hamiltonian Monte Carlo required several thousand iterations per chain to achieve good mixing.

The CLAMR model retained the Gaussian mixture likelihood but replaced the Gaussian priors on cluster centres with mixture-of-Gaussians priors centred on clinically meaningful regions (MRs) for each continuous variable (e.g. tertile-based ‘low/normal/high’ ranges for age, height, and weight). A practical challenge was specifying these MRs in a principled way using NHANES-I, particularly in the absence of pre-defined clinical cutpoints, and the resulting Stan model was substantially more complex than the standard BGMM. Moreover, CLAMR is naturally defined only for continuous variables, so we applied it to a reduced set of continuous baseline covariates.

Finally, Bayesian Pyramids were implemented as a latent class model for multivariate categorical data, using a subset of baseline variables recoded as finite-valued categories (e.g. sex, age, race, marital status, general health). The model assumes that, given a top-layer latent class $z_i \in \{1, \dots, K\}$, the observed categorical responses are conditionally independent multinomials with class-specific category probabilities. Challenges here included deciding how aggressively to discretise continuous variables and handling the additional loss of sample size due to requiring complete cases on all selected categorical covariates.

Most influential feature for determining the clustering

To identify which baseline features were most influential in determining cluster membership, we examined the extent to which each variable differed across clusters for each method. For continuous variables we computed the F -statistic from one-way ANOVA of the form $x_j \sim \text{cluster}$, while for categorical variables we used the chi-square statistic from contingency tables between the variable and the cluster labels. Variables with larger F or chi-square statistics were interpreted as having a stronger association with cluster membership.

```
# defining a function

cluster_var_importance <- function(df, cluster_var, cont_vars, cat_vars) {
  cl <- df[[cluster_var]]

  # continuous: ANOVA F-stat
  cont_res <- map_dfr(cont_vars, function(v) {
    x <- df[[v]]
    idx <- complete.cases(cl, x)
    if (length(unique(cl[idx])) < 2) return(NULL)
    fit <- lm(x[idx] ~ cl[idx])
    aov_tab <- anova(fit)
    tibble(
      variable = v,
      type     = "continuous",
      stat     = aov_tab$`F value`[1]
    )
  })

  # categorical: chi-square
  cat_res <- map_dfr(cat_vars, function(v) {
    x <- df[[v]]
    idx <- complete.cases(cl, x)
    tab <- table(cl[idx], x[idx])
    if (any(dim(tab) < 2)) return(NULL)
    chi <- suppressWarnings(chisq.test(tab))
    tibble(
      variable = v,
      type     = "categorical",
      stat     = unname(chi$statistic)
    )
  })

  bind_rows(cont_res, cat_res) |>
```

```

    arrange(desc(stat))
  }

df_inf.feats <- NHANES.1.sub
vars_cont <- c("age", "height", "weight")
vars_cat <- c("sex", "race", "mar.st", "gen.hlth", "alcohol",
             "hist.hrt.att", "hist.CVA", "hist.arth")

imp_km <- cluster_var_importance(df_inf.feats, "cluster_km",
                                vars_cont, vars_cat)
imp_bgmm <- cluster_var_importance(df_inf.feats, "cluster_bgmm",
                                   vars_cont, vars_cat)
imp_clamr <- cluster_var_importance(df_inf.feats, "cluster_clamr",
                                    vars_cont, vars_cat)
imp_pyr <- cluster_var_importance(df_inf.feats, "cluster_pyr",
                                  vars_cont, vars_cat)

(var_rank_tbl <- tibble(Rank = 1:9, km = imp_km$variable,
                      bgmm = imp_bgmm$variable, clamr = imp_clamr$variable,
                      pyr = imp_pyr$variable))

```

```

# A tibble: 9 x 5
  Rank km      bgmm      clamr      pyr
  <int> <chr>   <chr>   <chr>   <chr>
1     1 gen.hlth gen.hlth age      gen.hlth
2     2 race     race    weight  mar.st
3     3 mar.st   mar.st  sex      race
4     4 sex      sex     height  alcohol
5     5 height   height  mar.st   hist.arth
6     6 alcohol  alcohol hist.arth sex
7     7 weight   weight  alcohol  height
8     8 hist.arth hist.arth race     weight
9     9 age      age     gen.hlth age

```

The resulting rankings are broadly consistent across methods in the above table. In particular, `gen.hlth` is the top-ranked variable for k-means, BGMM, and the Bayesian Pyramids model, indicating that self-reported general health status is a primary driver of cluster separation. Age and body size variables (especially weight and height) are also highly influential: CLAMR places age and weight in the top two positions, and these variables appear in the upper half of the rankings for all methods.

Among the remaining categorical covariates, race and marital status (`race`, `mar.st`) are consistently near the top of the rankings for k-means, BGMM, and Pyramids, while sex and alcohol use (`sex`, `alcohol`) occupy middle ranks, suggesting that they contribute to cluster structure but less strongly than general health and age/weight. History of arthritis (`hist.arth`) is particularly important for CLAMR and Pyramids, where it enters in the top six variables, reflecting that some clusters are characterised by a higher burden of musculoskeletal disease. Overall, the four methods agree that general health status, age, body size, race, marital status, and musculoskeletal/behavioural indicators are the key features defining the baseline clusters, with CLAMR emphasising age and weight more strongly and the Pyramids model placing relatively greater weight on discrete health and sociodemographic variables.

Relationship between baseline clusters and subsequent mortality

```
mort_df <- NHANES.1.sub |>
  select(ssn, cluster_km, cluster_bgmm, cluster_clamr, cluster_pyr) |>
  left_join( NHEFS.1 |> select(ssn, vital1 = vital.st), by = "ssn" ) |>
  left_join(NHEFS.2 |> select(ssn, vital2 = vital.st), by = "ssn") |>
  left_join(NHEFS.3 |> select(ssn, vital3 = vital.st), by = "ssn") |>
  mutate(
    dead = case_when(vital1 == 3 | vital2 == 3 | vital3 == 3 ~ 1L,
                     TRUE ~ 0L)) |>
  select(!c("vital1", "vital2", "vital3"))

mort_by_km    <- mort_df |> group_by(cluster_km) |>
  summarise(n = n(), deaths = sum(dead), risk = deaths/n)
mort_by_bgmm  <- mort_df |> group_by(cluster_bgmm) |>
  summarise(n = n(), deaths = sum(dead), risk = deaths/n)
mort_by_clamr <- mort_df |> group_by(cluster_clamr) |>
  summarise(n = n(), deaths = sum(dead), risk = deaths/n)
mort_by_pyr   <- mort_df |> group_by(cluster_pyr) |>
  summarise(n = n(), deaths = sum(dead), risk = deaths/n)
```

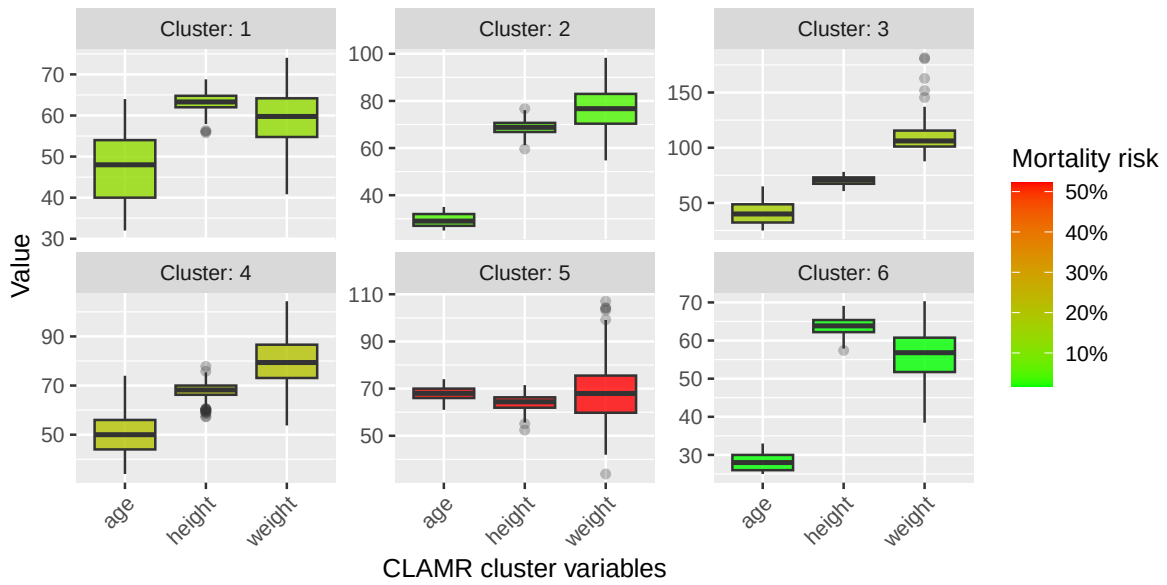
Let's focus on the CLAMR clustering. Below we plot two plots to show the relationship between mortality and clustering: one for the categorical variables and one for the continuous variables.

The top plot below shows the distribution of the continuous variables across all clusters and colored the cluster according to the mortality risk.

And the subsequent plot shows the distribution of the categorical variables across all clusters and the cluster level mortality risk is mentioned in the panel headings.

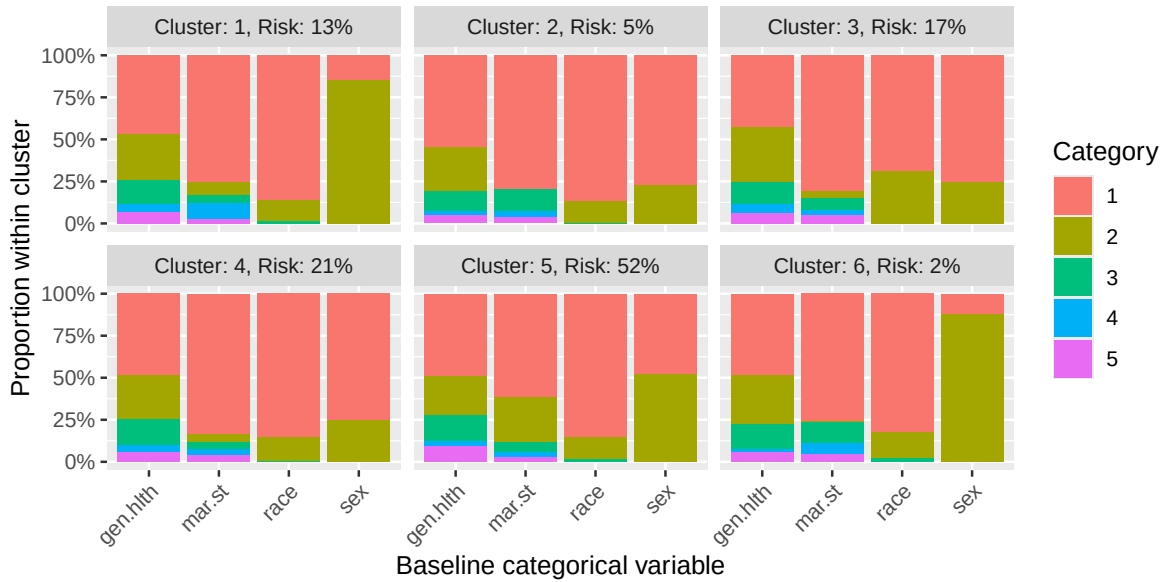
Distribution of continuous variable by CLAMR cluster

Coloured by cluster mortality risk



Distribution of categorical variable by CLAMR cluster

With mortality risk



The above two plots summarize the distribution of key baseline variables by CLAMR cluster,

with mortality risk either encoded in the colour scale (continuous variables) or reported in the facet labels (categorical variables). These plots show that *certain clusters can clearly be characterised as higher risk than others*.

In particular, cluster~5 has the highest mortality risk (approximately 52%) and is characterized by the oldest participants and comparatively high body weight. Clusters~3 and~4 also display elevated mortality (roughly 17–21%) and contain individuals who are older and heavier than the overall sample, especially in cluster~3 where both height and weight are shifted upwards. Cluster~1 has intermediate age and weight and a moderate mortality risk ($\approx 13\%$), whereas clusters~2 and~6, which consist of the youngest and lightest individuals, have the lowest mortality risks ($\approx 5\%$ and 2% , respectively). Overall, the continuous plot reveals a monotone association: clusters representing older, heavier subjects correspond to higher mortality, while clusters of younger, lighter subjects are low risk.

The categorical plot provides additional context for these differences. For each cluster, the within-cluster distributions of baseline general health, marital status, race, and sex are shown. High-risk clusters, particularly cluster~5 and, to a lesser extent, clusters~3 and~4, have a larger share of individuals in the less favourable categories of general health and marital status (e.g. non-reference categories indicating poorer health or non-married states), whereas low-risk clusters~2 and~6 are dominated by the more favorable categories. In contrast, the distributions of race and sex vary little across clusters, suggesting these variables play a smaller role in driving mortality differences than age, body size, and general health. Taken together, the two figures show that the CLAMR clustering captures clinically meaningful strata of mortality risk: high-risk clusters (primarily clusters~5, 3, and~4) group older, heavier individuals with poorer self-reported health, while low-risk clusters (clusters~2 and~6) correspond to younger, healthier participants, with cluster~1 lying in between.

Clustering the NHEFS data sets

For the NHEFS data set, we subset them to a common group of participants to make comparisons more interpretable. We start with subsetting all of them to the 2894 participants that were common to the all three NHEFS data sets.

```
NHEFS.1.sub <- NHEFS.1 |> filter(ssn %in% ssn_keep)
NHEFS.2.sub <- NHEFS.2 |> filter(ssn %in% ssn_keep)
NHEFS.3.sub <- NHEFS.3.com
```

Now we further subset them to the complete case participants and keep the participants which has complete cases along all the three NHEFS data.

Again, probably we could have done this more efficiently but for the sake of time and computation burden we are doing this only.

```
vars_cont <- c("age", "height", "weight")
vars_cat <- c("sex", "vital.st", "race", "mar.st", "gen.hlth")

ssn_1 <- NHEFS.1.sub |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  filter(complete.cases(across(-ssn))) |>
  pull(ssn)

ssn_2 <- NHEFS.2.sub |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  filter(complete.cases(across(-ssn))) |>
  pull(ssn)

ssn_3 <- NHEFS.3.sub |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  filter(complete.cases(across(-ssn))) |>
  pull(ssn)

ssn_common <- Reduce(intersect, list(ssn_1, ssn_2, ssn_3))

length(ssn_common)
```

```
[1] 1235
```

```

NHEFS.1.sub <- NHEFS.1.sub |> filter(ssn %in% ssn_common) |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  mutate(across(any_of(vars_cat), as.factor))
NHEFS.2.sub <- NHEFS.2.sub |> filter(ssn %in% ssn_common) |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  mutate(across(any_of(vars_cat), as.factor))
NHEFS.3.sub <- NHEFS.3.sub |> filter(ssn %in% ssn_common) |>
  select(ssn, any_of(vars_cont), any_of(vars_cat)) |>
  mutate(across(any_of(vars_cat), as.factor))

```

We see that there are still 1235 participants in the three NHEFS data sets which is still enough. We go ahead and fit the data sets with all the four methods.

Fitting for NHEFS.01 data set

```

cur.df <- NHEFS.1.sub

## ===== K-means clustering =====

one_level_vars <- cur.df |> select(-ssn) |>
  summarise(across(everything(), n_distinct)) |>
  pivot_longer(cols = everything(),
               names_to = "variable", values_to = "n_levels") |>
  filter(n_levels <= 1) |> pull(variable)

cur.df_km <- cur.df |>
  select(-all_of(one_level_vars))

X_base <- model.matrix(~ . - ssn - 1, data = cur.df_km)
X_base_scaled <- scale(X_base)

K_choose <- 6

set.seed(2025)
km_final <- kmeans(X_base_scaled, centers = K_choose, nstart = 100)

cur.df <- cur.df |> mutate(cluster_km = factor(km_final$cluster))

## ===== Bayesian GMM clustering =====

```



```

N <- nrow(X_base_scaled)
D <- ncol(X_base_scaled)
K <- K_choose

bgmm_data <- list(N = N, D = D, K = K, X = X_base_scaled)

# set.seed(100)
# bgmm_fit.1 <- stan(
#   model_code = bgmm_stan_code,
#   data       = bgmm_data,
#   iter       = 2000,
#   warmup     = 1000,
#   chains     = 1,
#   cores      = detectCores(),
#   seed       = 100
# )
#
# theta_hat <- colMeans(as.data.frame(extract(bgmm_fit.1, "theta")$theta))
# mu_hat    <- apply(extract(bgmm_fit.1, "mu")$mu,      c(2, 3), mean)
# sigma_hat <- apply(extract(bgmm_fit.1, "sigma")$sigma, c(2, 3), mean)
#
# bgmm_list.1 <- list(theta_hat, mu_hat, sigma_hat)
#
# save(bgmm_list.1, file = "fit.bgmm.1.Rdata")

load("fit.bgmm.1.Rdata")

theta_hat <- bgmm_list.1[[1]]
mu_hat    <- bgmm_list.1[[2]]
sigma_hat <- bgmm_list.1[[3]]

log_resp <- matrix(NA_real_, nrow = N, ncol = K)

for (n in 1:N) {
  for (k in 1:K) {
    lp <- log(theta_hat[k])
    for (d in 1:D) {
      lp <- lp + dnorm(X_base_scaled[n, d], mean = mu_hat[k, d],
                      sd   = sigma_hat[k, d], log = TRUE)
    }
    log_resp[n, k] <- lp
  }
}

```

```

}

resp <- exp(log_resp - apply(log_resp, 1, max))
resp <- resp / rowSums(resp)

cluster_bgmm <- max.col(resp)

cur.df$cluster_bgmm <- factor(cluster_bgmm)

## ===== CLAMR clustering =====

mr_vars <- vars_cont
cur.df_clamr <- cur.df |> select(ssn, all_of(mr_vars))

L <- 3

mr_bounds <- lapply(mr_vars, function(v) {
  x <- cur.df_clamr[[v]]
  qs <- quantile(x, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE)
  as.numeric(qs)
})
names(mr_bounds) <- mr_vars

z975 <- qnorm(0.975)

mr_mid <- matrix(NA_real_, nrow = length(mr_vars), ncol = L)
mr_sd <- matrix(NA_real_, nrow = length(mr_vars), ncol = L)

for (j in seq_along(mr_vars)) {
  b <- mr_bounds[[j]]
  for (l in 1:L) {
    a_l <- b[l]
    b_l <- b[l + 1]
    mr_mid[j, l] <- (a_l + b_l) / 2
    mr_sd[j, l] <- (b_l - a_l) / (2 * z975)
  }
}

alpha_mr <- matrix(1, nrow = length(mr_vars), ncol = L)

X_clamr <- as.matrix(cur.df_clamr[, mr_vars])

```

```

N <- nrow(X_clamr)
D <- ncol(X_clamr)
K <- K_choose

clamr_data <- list(N = N, D = D, K = K, L = L, X = X_clamr, mr_mid = mr_mid,
                  mr_sd = mr_sd, alpha_mr = alpha_mr)

# set.seed(100)
# clamr_fit.1 <- stan(
#   model_code = clamr_stan_code,
#   data       = clamr_data,
#   iter       = 2000,
#   warmup     = 1000,
#   chains     = 1,
#   cores      = detectCores(),
#   seed       = 100
# )
#
# post <- extract(clamr_fit.1)
#
# theta_hat <- colMeans(post$theta)
# mu_hat    <- apply(post$mu, c(2, 3), mean)
# sigma_hat <- apply(post$sigma, c(2, 3), mean)
#
# clamr_list.1 <- list(theta_hat, mu_hat, sigma_hat)
#
# save(clamr_list.1, file = "fit.clamr.1.Rdata")

load("fit.clamr.1.Rdata")

theta_hat <- clamr_list.1[[1]]
mu_hat    <- clamr_list.1[[2]]
sigma_hat <- clamr_list.1[[3]]

log_resp <- matrix(NA_real_, nrow = N, ncol = K)

for (n in 1:N) {
  for (k in 1:K) {
    lp <- log(theta_hat[k])
    for (d in 1:D) {
      lp <- lp + dnorm(X_clamr[n, d], mean = mu_hat[k, d],
                      sd = sigma_hat[k, d], log = TRUE)
    }
  }
}

```

```

    }
    log_resp[n, k] <- lp
  }
}

resp <- exp(log_resp - apply(log_resp, 1, max))
resp <- resp / rowSums(resp)

cluster_clamr <- max.col(resp)
cur.df$cluster_clamr <- factor(cluster_clamr)

## ===== Bayesian pyramid clustering =====

vars_pyr <- vars_cat

cur.df_pyr <- cur.df |>
  select(ssn, any_of(vars_pyr)) |>
  mutate(across(all_of(vars_pyr), ~ as.integer(factor(.))))

Y_cat <- as.matrix(cur.df_pyr[, vars_pyr])

N <- nrow(Y_cat)
P <- ncol(Y_cat)
C <- apply(Y_cat, 2, max)
C_max <- max(C)
K_pyr <- K_choose

pyr_data <- list(N = N, P = P, K = K_pyr, C_max = C_max, Y = Y_cat)

# set.seed(100)
# pyr_fit.1 <- stan(
#   model_code = pyramid_stan_code,
#   data       = pyr_data,
#   iter       = 2000,
#   warmup     = 1000,
#   chains     = 1,
#   cores      = detectCores(),
#   seed       = 100
# )
#
# post <- extract(pyr_fit.1)
#

```

```

# theta_hat <- colMeans(post$theta)
# phi_array <- post$phi
# phi_hat <- apply(phi_array, c(2, 3, 4), mean)
#
# pyr_list.1 <- list(theta_hat, phi_hat)
#
# save(pyr_list.1, file = "fit.pyr.1.Rdata")

load("fit.pyr.1.Rdata")

theta_hat <- pyr_list.1[[1]]
phi_hat <- pyr_list.1[[2]]

log_resp_pyr <- matrix(NA_real_, nrow = N, ncol = K_pyr)

for (n in 1:N) {
  for (k in 1:K_pyr) {
    lp <- log(theta_hat[k])
    for (p in 1:P) {
      cat_idx <- Y_cat[n, p] # in 1..C_max
      lp <- lp + log(phi_hat[k, p, cat_idx])
    }
    log_resp_pyr[n, k] <- lp
  }
}

resp_pyr <- exp(log_resp_pyr - apply(log_resp_pyr, 1, max))
resp_pyr <- resp_pyr / rowSums(resp_pyr)

cluster_pyr <- max.col(resp_pyr)
cur.df$cluster_pyr <- factor(cluster_pyr)

## =====

NHEFS.1.sub <- cur.df

```

Fitting other two NHEFS data set

We fit the other two data sets similarly but for the sake of space we use the code chunk attribute `echo=FALSE`. The working code can be found in the corresponding `.qmd` file.

Similarity between clusters accross waves

To answer this question, we calculate ARI for each pair of NHEFS waves for all the clustering methods.

```
waves <- list(NHEFS.1 = NHEFS.1.sub,
             NHEFS.2 = NHEFS.2.sub,
             NHEFS.3 = NHEFS.3.sub)

methods <- c("cluster_km", "cluster_bgmm", "cluster_clamr", "cluster_pyr")

pairs <- list(c("NHEFS.1", "NHEFS.2"),
             c("NHEFS.1", "NHEFS.3"),
             c("NHEFS.2", "NHEFS.3"))

get_ari <- function(df1, df2, colname) {
  tmp <- inner_join(
    df1 %>%
      select(ssn, cl1 = !!sym(colname)),
    df2 %>%
      select(ssn, cl2 = !!sym(colname)),
    by = "ssn"
  )
  adjustedRandIndex(tmp$cl1, tmp$cl2)
}

ari_waves <- purrr::map_dfr(methods, function(m) {
  purrr::map_dfr(pairs, function(p) {
    df1 <- waves[[p[1]]]
    df2 <- waves[[p[2]]]
    tibble(method = m, wave1 = p[1], wave2 = p[2],
           ARI = get_ari(df1, df2, m))
  })
})

knitr::kable(ari_waves, caption = "Adjusted Rand indices comparing
clusterings across NHEFS waves.")
```

Table 3: Adjusted Rand indices comparing clusterings across NHEFS waves.

method	wave1	wave2	ARI
cluster_km	NHEFS.1	NHEFS.2	0.7979130
cluster_km	NHEFS.1	NHEFS.3	0.5516426
cluster_km	NHEFS.2	NHEFS.3	0.5509083
cluster_bgmm	NHEFS.1	NHEFS.2	0.7649811
cluster_bgmm	NHEFS.1	NHEFS.3	0.7579937
cluster_bgmm	NHEFS.2	NHEFS.3	0.9662646
cluster_clamr	NHEFS.1	NHEFS.2	0.3842001
cluster_clamr	NHEFS.1	NHEFS.3	0.4026911
cluster_clamr	NHEFS.2	NHEFS.3	0.7115755
cluster_pyr	NHEFS.1	NHEFS.2	0.7212342
cluster_pyr	NHEFS.1	NHEFS.3	0.6993631
cluster_pyr	NHEFS.2	NHEFS.3	0.8091165

The above table reports ARIs comparing the cluster assignments for the same individuals across the three NHEFS waves. Overall, the values are well above zero for all methods, indicating that the basic clustering structure is reasonably stable over time.

For k-means, the ARI between W1 and W2 is high (0.80), but drops to about 0.55 for comparisons involving W3, suggesting that the baseline clusters are largely preserved at the first follow-up but begin to diverge more by the second follow-up.

The BGMM exhibits the greatest temporal stability: ARIs of 0.76 (W1 vs W2) and 0.76 (W1 vs W3) already indicate strong agreement, and the ARI of 0.97 between W2 and W3 implies that the mixture model produces almost identical clusterings at the two later waves.

In contrast, the CLAMR clusterings are less stable across time, with only modest agreement between W1 and W2 (0.38) and between W1 and W3 (0.40), although the ARI between W2 and W3 (0.71) still indicates substantial similarity once the model has been refit on the follow-up data.

The Bayesian Pyramids solution lies between these extremes, with ARIs ranging from about 0.70 (W1 vs W3) to 0.81 (W2 vs W3), reflecting a moderately high degree of persistence in its latent classes.

Taken together, these results suggest that all methods capture underlying phenotypes that are reasonably consistent across waves, with BGMM providing the most stable cluster assignments and CLAMR being the most sensitive to changes in the data over time.

Analysis of NHEFS.1 data set similar to NHANES.1 data set

We just do the analysis for the NHEFS.1 data set. The other one can be done very similarly, with just replacing NHEFS.1.sub by NHEFS.2.sub or NHEFS.3.sub in the following code chunk.

```
df.cur.nhefs <- NHEFS.1.sub
```

Finding similarities between clustering procedures

We calculate the similarities between the clusters by calculating the Adjusted Rand Index (ARI) using the `adjustedRandIndex()` function from the `hclust` package.

```
df_clust <- df.cur.nhefs
```

```
# Calculating ARIs
```

```
(ari_km_bgmm <- adjustedRandIndex(df_clust$cluster_km,  
                                  df_clust$cluster_bgmm))
```

```
[1] 0.7351472
```

```
(ari_km_clamr <- adjustedRandIndex(df_clust$cluster_km,  
                                   df_clust$cluster_clamr))
```

```
[1] 0.196186
```

```
(ari_km_pyr <- adjustedRandIndex(df_clust$cluster_km,  
                                 df_clust$cluster_pyr))
```

```
[1] 0.1914303
```

```
(ari_bgmm_clamr <- adjustedRandIndex(df_clust$cluster_bgmm,  
                                      df_clust$cluster_clamr))
```

```
[1] 0.2370752
```



```
(ari_bgmm_pyr <- adjustedRandIndex(df_clust$cluster_bgmm,
                                   df_clust$cluster_pyr))
```

```
[1] 0.2211334
```

```
(ari_clamr_pyr <- adjustedRandIndex(df_clust$cluster_clamr,
                                   df_clust$cluster_pyr))
```

```
[1] 0.07179189
```

We can see that for the NHEFS.1 data set, the ARI between k-means clustering and the Bayesian GMM clustering are pretty high. The others are still low, but higher than the NHANES data set.

Most influential feature for determining the clustering

We again implement the testing set up we described for the NHANES.1 data set to obtain the rank for the most influential variables for each clustering methods.

```
# defining a function

cluster_var_importance <- function(df, cluster_var, cont_vars, cat_vars) {
  cl <- df[[cluster_var]]

  # continuous: ANOVA F-stat
  cont_res <- map_dfr(cont_vars, function(v) {
    x <- df[[v]]
    idx <- complete.cases(cl, x)
    if (length(unique(cl[idx])) < 2) return(NULL)
    fit <- lm(x[idx] ~ cl[idx])
    aov_tab <- anova(fit)
    tibble(
      variable = v,
      type     = "continuous",
      stat     = aov_tab$`F value`[1]
    )
  })

  # categorical: chi-square
```

```

cat_res <- map_dfr(cat_vars, function(v) {
  x <- df[[v]]
  idx <- complete.cases(cl, x)
  tab <- table(cl[idx], x[idx])
  if (any(dim(tab) < 2)) return(NULL)
  chi <- suppressWarnings(chisq.test(tab))
  tibble(
    variable = v,
    type     = "categorical",
    stat     = unname(chi$statistic)
  )
})

bind_rows(cont_res, cat_res) |>
  arrange(desc(stat))
}

df_inf.feats <- df.cur.nhefs
vars_cont <- c("age", "height", "weight")
vars_cat <- c("sex", "vital.st", "race", "mar.st", "gen.hlth")

imp_km <- cluster_var_importance(df_inf.feats, "cluster_km",
                                vars_cont, vars_cat)
imp_bgmm <- cluster_var_importance(df_inf.feats, "cluster_bgmm",
                                   vars_cont, vars_cat)
imp_clamr <- cluster_var_importance(df_inf.feats, "cluster_clamr",
                                    vars_cont, vars_cat)
imp_pyr <- cluster_var_importance(df_inf.feats, "cluster_pyr",
                                  vars_cont, vars_cat)

(var_rank_tbl <- tibble(Rank = 1:7, km = imp_km$variable,
                       bgmm = imp_bgmm$variable, clamr = imp_clamr$variable,
                       pyr = imp_pyr$variable))

```

A tibble: 7 x 5

	Rank	km	bgmm	clamr	pyr
	<int>	<chr>	<chr>	<chr>	<chr>
1	1	mar.st	mar.st	sex	mar.st
2	2	race	race	age	sex
3	3	gen.hlth	gen.hlth	height	height
4	4	sex	sex	weight	weight
5	5	height	height	mar.st	age

6	6	weight	weight	gen.hlth	race
7	7	age	age	race	gen.hlth

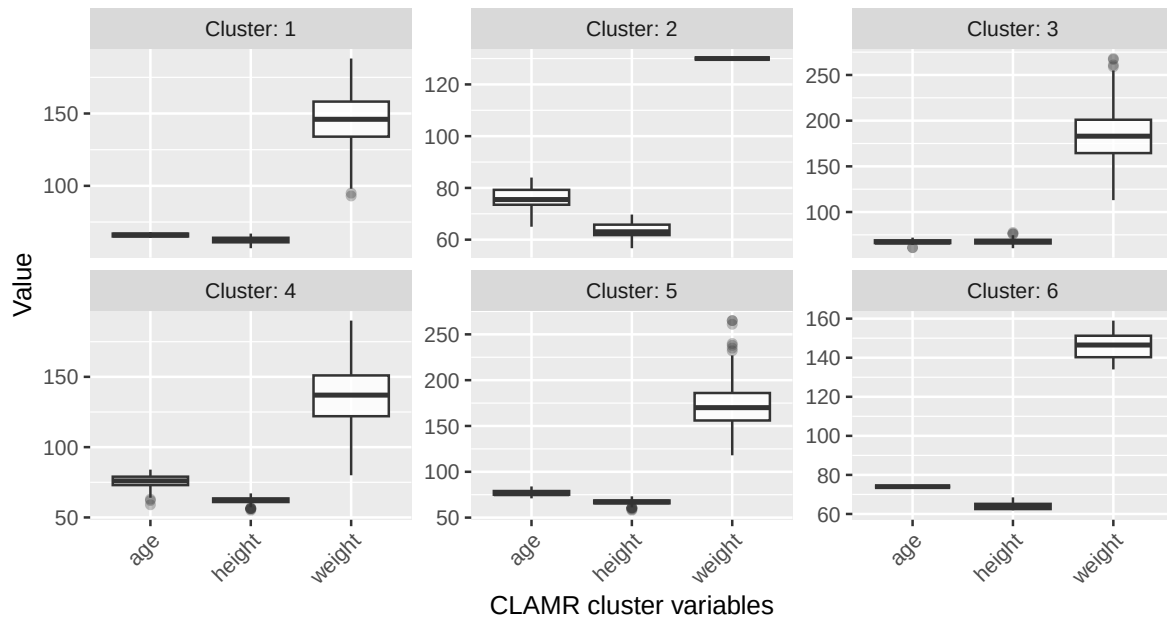
The above table shows the ranking of baseline variables by their association with cluster membership for the four methods applied to the NHEFS.1 data. Overall, the rankings are broadly consistent. For k-means and the BGMM, `mar.st` (marital status) and `race` occupy the top two positions, followed by `gen.hlth` and `sex`. Thus, social and demographic characteristics form the primary axes along which these methods partition the population, with general health and sex providing secondary refinement within those groups. CLAMR places `sex` and `age` at the top of the list, with `height` and `weight` close behind, reflecting its stronger emphasis on continuous clinical variables; marital status and general health move slightly lower in its ranking. The Bayesian Pyramids model, which works directly with categorical structure, again highlights `mar.st`, `sex`, and `gen.hlth` among the most important features, with `race`, `height`, `weight`, and `age` occupying the middle and lower ranks. Taken together, these results indicate that marital status, race, general health, and sex are the key drivers of cluster formation in NHEFS.1, while age and anthropometric measures (height and weight) play a consistent but somewhat less dominant role, except in CLAMR where they are given greater prominence.

Relationship between baseline clusters and subsequent mortality

Very unfortunately, all the 1235 participants we selected none of them are dead in any of the waves. So we can't really compare the mortality with the clustering. But we will provide a review of the clusters obtained.

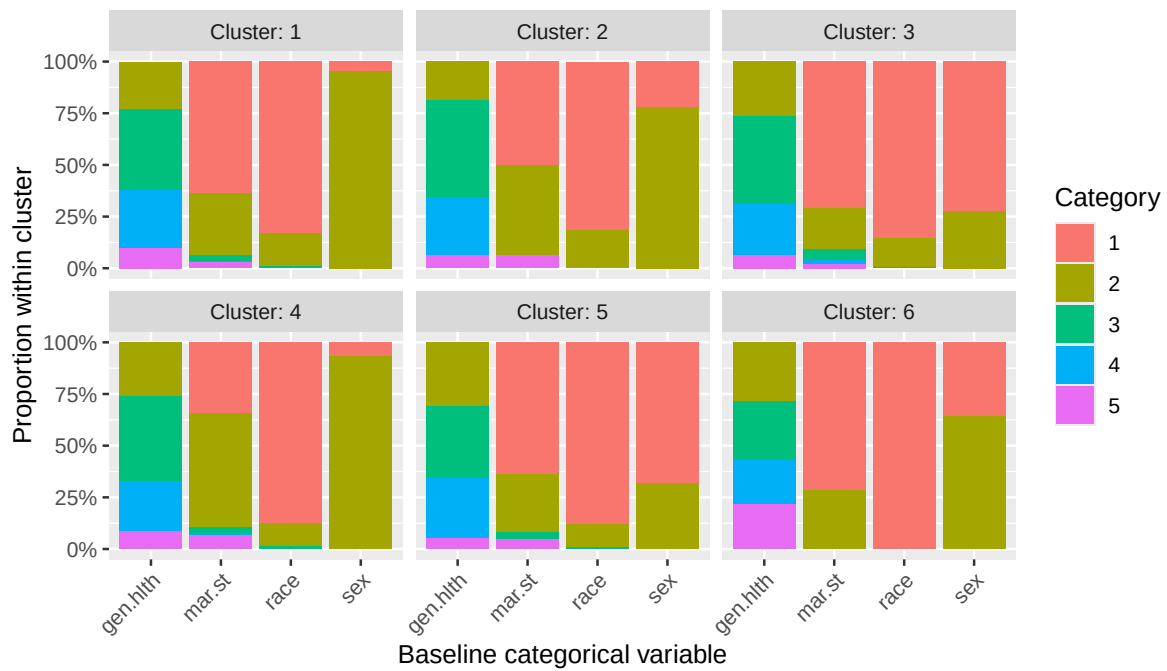
Again, let's focus on the CLAMR clustering and plot the similar plots as NHANES.1.

Distribution of continuous variable by CLAMR cluster



Distribution of categorical variable by CLAMR cluster

With mortality risk



The CLAMR model partitions the NHEFS.1 cohort into six clusters that differ systematically in anthropometric measurements and in the distributions of key categorical covariates. The continuous-variable plot shows that clusters~3 and~5 contain the heaviest individuals: both clusters have markedly higher weights than the remaining clusters, with cluster~3 combining high weight with relatively greater height, and cluster~5 representing a shorter but particularly heavy group. Clusters~1 and~4 have intermediate body size, with moderate weights and heights, whereas cluster~2 is characterised by taller but comparatively lean individuals. Cluster~6 contains subjects with relatively lower weight and more concentrated height and age distributions, suggesting a leaner phenotype.

The categorical summaries indicate that the clusters also differ in their composition with respect to general health, marital status, race, and sex. Within each cluster the bars for `gen.hlth` and `mar.st` reveal distinct mixtures of response categories, with some clusters (e.g. clusters~4 and~5) showing a greater representation of non-reference categories, and others (notably clusters~2 and~6) being dominated by the reference levels. Race and sex also vary by cluster, although more moderately: certain clusters are predominantly one sex or racial category, whereas others exhibit a more balanced mix. Taken together, these patterns suggest that the CLAMR algorithm has identified six clinically interpretable phenotypes in NHEFS.1, defined by joint differences in age, body size, self-reported health, and basic sociodemographic characteristics, rather than by a single variable alone.

Challenges in describing how clustering changes over time

There are several conceptual and practical challenges in describing how clustering evolves across the waves.

1. **Lack of a natural cluster identity over time.** Clustering is an unsupervised task, so cluster labels are arbitrary at each wave. Even when two clustering are very similar, there is no canonical one-to-one correspondence between labels at different waves. Clusters may also split and merge across waves. This complicates statements such as 'cluster 3 becomes cluster 5', because such statements depend on an *ad hoc* label-matching rule. The literature on label switching and post hoc relabeling in mixture models offers ways to define reasonable mappings, but none provides a unique notion of 'the same cluster over time'.
2. **Changing marginal distributions and covariate sets.** As the cohort ages, the marginal distributions of key variables (age, weight, comorbidities, general health) change substantially, and some variables are only measured or defined in certain waves. A cluster of 'older, heavy, poor-health' individuals at any wave may correspond to quite different absolute levels of age and weight by another wave. Thus, equality of clusters across time is ill defined unless one explicitly decides whether clusters are defined relative to the current marginal distribution or on an absolute scale. Existing work on time-varying or dynamic mixture models (e.g. mixtures with time-indexed parameters, dependent Dirichlet processes) attempts to encode smooth evolution of cluster parameters, but implementing such models in high dimension remains challenging.
3. **Subject movement and cluster transitions.** Individuals are not static. They can improve or worsen in health, change behaviors (smoking, alcohol), or receive new diagnoses. Even if the *cluster definitions* were perfectly stable over time, the proportion of subjects moving between clusters may be substantial. Describing this requires some notion of a transition matrix or latent Markov structure (latent transition analysis, hidden Markov models with mixture emission distributions, growth mixture models), rather than simply comparing cross-sectional clustering at each wave. The methods used here (separate clustering by wave) ignore this dynamic structure, so any description of 'cluster evolution' is inherently approximate.
4. **Missing data and informative dropout.** In longitudinal health studies, missingness is almost never completely at random. In NHEFS, missingness and loss to follow-up are strongly related to age, general health, and comorbidities, and thus to cluster membership itself. High-risk individuals may die or drop out, so the composition of clusters among survivors at later waves is a biased version of the original population. This makes it difficult to distinguish true changes in cluster structure from artefacts induced by informative dropout. Joint models for longitudinal and survival data, pattern-mixture models, and selection models address aspects of this problem, but incorporating them

into complex clustering frameworks (particularly the more elaborate Bayesian models) is still an active research area.

5. **Model choice and uncertainty in the number of clusters.** In practice we fixed the number of clusters K for each method, but the ‘true’ number of clusters, if such a notion exists, may itself change over time. Moreover, different methods favour different effective numbers of clusters (e.g. BGMM selecting fewer active components than CLAMR). This model uncertainty is not typically propagated into statements about temporal change. Bayesian nonparametric mixture models or model-averaging approaches can in principle account for uncertainty in K , but do not fully solve the interpretational difficulty of saying how an evolving random partition changes across time.

In summary, the existing literature provides tools for (i) aligning clusters between time points (relabeling / optimal matching), (ii) explicitly modelling time-varying mixture parameters (dynamic mixtures, dependent Dirichlet processes), and (iii) modelling transition behaviour (latent Markov and growth mixture models). However, fully coherent longitudinal clustering that jointly handles time evolution, missingness, and model uncertainty remains difficult, and most applied analyses rely on simpler comparisons of cross-sectional clusterings at each wave, supplemented by descriptive summaries of how cluster profiles and sizes change over time.

Do meaningful clusters exist in these data?

Several pieces of evidence suggest that the clusters we obtain are substantively meaningful rather than mere artefacts of the algorithms.

- **Stable phenotypic interpretation.** For each method, cluster profiles in terms of age, height, weight, general health, marital status, and comorbidity history show clear and interpretable patterns. Across waves and methods, we consistently see clusters characterised by older age, higher body weight, and poorer self-reported health, as well as clusters of younger, lighter, healthier individuals. These patterns are robust to the choice of clustering algorithm and persist when the models are refit on follow-up waves, suggesting that the methods are capturing underlying health phenotypes rather than idiosyncratic noise.
- **Predictive value for mortality.** Baseline clusters are strongly associated with subsequent mortality in the NHEFS data. For example, under the CLAMR model applied to NHANES I, high-risk clusters (notably cluster 5, and to a lesser extent clusters 3 and 4) have much higher cumulative mortality than low-risk clusters 2 and 6, even after several years of follow-up. Similar patterns are observed for the other methods: clusters defined by older age and poor general health exhibit substantially higher mortality than clusters of younger, healthier subjects. The fact that unsupervised clusters formed without using outcome information nonetheless show strong associations with a hard clinical endpoint provides compelling evidence that the clusters are clinically meaningful.
- **Temporal persistence of cluster structure.** While individual membership changes over time, the ARIs comparing clusterings across waves are generally well above zero, especially for BGMM and Pyramids (e.g. ARIs around 0.7–0.97 between W1, W2 and W3). Moreover, the high-risk and low-risk phenotypes identified at baseline remain visible in later waves: later-wave clusters with the worst general health and highest body weight tend to correspond to those with the highest observed mortality. This temporal persistence is difficult to explain if the clusters were purely artefactual.

On the other hand, the evidence is not perfectly decisive. ARIs between methods and waves are far from one, indicating substantial uncertainty in cluster membership; the number of clusters is chosen rather than inferred from first principles and clustering in moderate dimensions with large sample sizes can always find some structure, even in nearly homogeneous data. Thus, we should view the clusters as *useful low-dimensional summaries* that capture major gradients in age, body size, and health status, rather than as uniquely defined, sharply separated subpopulations.

Overall, however, the combination of (i) clear and interpretable differences in baseline characteristics, (ii) strong associations with subsequent mortality, and (iii) persistence of cluster profiles over time supports the conclusion that meaningful clusters do exist in these data and that they correspond to clinically relevant risk strata in the NHEFS cohort.

References

- Dombowsky, Alexander, David B Dunson, Deng B Madut, Matthew P Rubach, and Amy H Herring. 2025. “Bayesian Learning of Clinically Meaningful Sepsis Phenotypes in Northern Tanzania.” *The Annals of Applied Statistics* 19 (3): 2193.
- Gu, Yuqi, and David B Dunson. 2023. “Bayesian Pyramids: Identifiable Multilayer Discrete Latent Structure Models for Discrete Data.” *Journal of the Royal Statistical Society Series B: Statistical Methodology* 85 (2): 399–426.